



**Politechnika
Śląska**

PROJEKT INŻYNIERSKI

System wspomagający inteligentne składanie zestawów komputerowych

Piotr KUPCZYK

Nr albumu: 305198

Kierunek: Teleinformatyka

PROWADZĄCY PRACĘ

dr hab. inż. Bożena Małysiak-Mrozek

KATEDRA Systemów Rozproszonych i Urządzeń Informatyki

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2026

Tytuł pracy

System wspomagający inteligentne składanie zestawów komputerowych

Streszczenie

Celem pracy było zaprojektowanie i zaimplementowanie inteligentnego systemu webowego wspomagającego składanie zestawów komputerowych z wykorzystaniem logiki rozmytej. Opracowana aplikacja umożliwia dobór kompatybilnych komponentów PC, weryfikując m.in. zgodność gniazda procesora, standardu pamięci RAM, wymagania energetyczne oraz dopasowanie rozmiarów karty graficznej i układu chłodzenia. Architektura systemu obejmuje front-end webowy (React, TypeScript, Tailwind CSS) komunikujący się przez API REST z backendem (Python, FastAPI, SQLAlchemy) korzystającym z bazy danych PostgreSQL, a rozwiązanie zostało wdrożone w chmurze Microsoft Azure. W systemie zastosowano elementy logiki rozmytej do klasyfikacji proponowanych konfiguracji komputerowych (np. budżetowych, energooszczędnych) na podstawie wielu parametrów jednocześnie. System ułatwia użytkownikom efektywne komponowanie technicznie zgodnych i dopasowanych do potrzeb zestawów PC.

Słowa kluczowe

aplikacja webowa, system wspomagania decyzji, konfiguracja zestawów komputerowych, kompatybilność podzespołów komputerowych, logika rozmyta, Python, aplikacje chmurowe, architektura klient-serwer, baza danych relacyjna

Thesis title

A system supporting intelligent assembly of computer sets

Abstract

The aim of this thesis was to design and implement an intelligent web-based system to assist in assembling custom computer sets using fuzzy logic. The developed application allows users to select compatible PC components by verifying, among others, CPU socket compatibility, RAM standards, power requirements, and ensuring components fit within the PC case dimensions. The system's architecture consists of a web front end (React, TypeScript, Tailwind CSS) communicating via a REST API with a back end (Python, FastAPI, SQLAlchemy) that utilizes a PostgreSQL database, and the solution is deployed on the Microsoft Azure cloud infrastructure. The system applies fuzzy logic techniques to classify proposed computer configurations (e.g., budget or energy-efficient builds) based on multiple criteria simultaneously. The system helps users efficiently compose PC sets that are technically compatible and tailored to their needs.

Key words

web application, decision support system, computer system configuration, hardware compatibility, fuzzy logic, Python, cloud applications, client-server architecture, relational database

Spis treści

1	Wstęp	1
1.1	Wprowadzenie do problemu	1
1.2	Osadzenie problemu w dziedzinie	1
1.3	Cel pracy	1
1.4	Zakres prac	2
1.5	Charakterystyka rozdziałów	3
1.6	Wkład autora	4
2	Analiza problemu oraz przegląd dostępnych rozwiązań	5
2.1	Sformułowanie problemu	5
2.2	Analiza istniejących rozwiązań w najpopularniejszych sklepach na polskim rynku	6
2.3	Porównanie istniejących rozwiązań z rozwiązaniem inżynierskim	6
2.4	Logika rozmyta w systemach wspomagania decyzji	7
2.5	Architektura systemu teleinformatycznego aplikacji webowej	9
2.6	Środowisko chmurowe jako środowisko wykonawcze systemu	9
2.6.1	Bezpieczeństwo systemu	11
3	Wymagania i narzędzia	13
3.1	Wymagania funkcjonalne	13
3.2	Wymagania нефункционалне	15
3.3	Przykładowe scenariusze użycia systemu	15
3.4	Narzędzia i technologie	16
3.4.1	Frontend	16
3.4.2	Backend	16
3.4.3	Modelowanie i narzędzia wspomagające	17
3.5	Metodyka pracy	17
4	Specyfikacja zewnętrzna	19
4.1	Sposób instalacji systemu	19
4.1.1	Uruchomienie systemu w środowisku lokalnym	19

4.1.2	Uruchomienie systemu w środowisku chmurowym Azure	20
4.2	Kategorie użytkowników	20
4.3	Sposób obsługi systemu	21
4.4	Administracja systemem	21
4.5	Kwestie bezpieczeństwa	22
4.6	Scenariusze korzystania z systemu	22
4.6.1	Scenariusz 1: Przeglądanie katalogu komponentów	23
4.6.2	Scenariusz 2: Filtrowanie oraz wybór produktów	23
4.6.3	Scenariusz 3: Dodawanie produktów do koszyka	24
4.6.4	Scenariusz 4: Poprawne logowanie użytkownika	25
4.6.5	Scenariusz 5: Niepoprawne logowanie użytkownika	26
4.6.6	Scenariusz 6: Utworzenie kompatybilnego zestawu komputerowego	26
4.6.7	Scenariusz 7: Próba utworzenia niekompatybilnego zestawu	28
5	Specyfikacja wewnętrzna	29
5.1	Przedstawienie idei systemu	29
5.1.1	Cel powstania aplikacji	29
5.1.2	Innowacyjność systemu	30
5.2	Architektura systemu	30
5.2.1	Schemat ideowy	30
5.2.2	Frontend	33
5.2.3	Backend	34
5.2.4	Baza danych	35
5.2.5	Infrastruktura chmurowa	37
5.3	Baza danych	39
5.3.1	Model danych - ERD	40
5.3.2	Opis najważniejszych encji	43
5.4	Komponenty i moduły systemu	56
5.4.1	Moduł kompatybilności podzespołów	57
5.4.2	Moduł logiki rozmytej	57
5.4.3	Moduł zarządzania koszykiem i zamówieniami	59
5.5	Najważniejsze algorytmy	60
5.6	Diagramy UML i realizacja przypadków użycia	61
5.6.1	Rejestracja użytkownika	61
5.6.2	Logowanie użytkownika	62
5.6.3	Wyświetlanie katalogu komponentów	62
5.6.4	Budowa zestawu komputerowego	63
5.6.5	Weryfikacja kompatybilności zestawu	64
5.6.6	Dodanie zestawu do koszyka	65

6	Weryfikacja i walidacja	67
6.1	Cel weryfikacji systemu	67
6.2	Metody testowania	67
6.3	Organizacja eksperymentów	68
6.4	Przypadki testowe	69
6.5	Wyniki testów	70
6.6	Wykryte i usunięte błędy	70
6.7	Ocena poprawności działania systemu	71
7	Podsumowanie i wnioski	73
7.1	Realizacja celów pracy	73
7.2	Ocena otrzymanych rezultatów	73
7.3	Trudności napotkane podczas realizacji	74
7.4	Możliwe kierunki dalszego rozwoju	74
7.5	Wnioski końcowe	74
	Bibliografia	77
	Spis skrótów i symboli	81
	Spis rysunków	83
	Spis tabel	85

Rozdział 1

Wstęp

Niniejszy rozdział stanowi wprowadzenie do tematyki pracy inżynierskiej. Przedstawiono w nim motywy realizacji tematu oraz zarys problemu, którego rozwiązanie stanowi przedmiot dalszych rozważań. W rozdziale określono również zakres pracy oraz jej główne cele.

1.1 Wprowadzenie do problemu

Sklepy komputerowe zazwyczaj posiadają swój własny sklep internetowy, oferujący możliwość zakupów części indywidualnie, jak i gotowych zestawów PC. Odpowiedni dobór komponentów jest istotną częścią procesu projektowania własnego stanowiska, a pomimo tego najwięksi sprzedawcy nie oferują możliwości sprawdzania kompatybilności wybranych produktów. Dlatego projekt ma na celu utworzenie prostego i intuicyjnego systemu internetowego, który zaoferuje przystępną formę weryfikacji poprawności wyboru komponentów z katalogu oraz finalnie dodanie kompatybilnego zestawu do koszyka użytkownika.

1.2 Osadzenie problemu w dziedzinie

Projekt osadzony jest w dziedzinie inżynierii oprogramowania, projektowania baz danych oraz systemów e-commerce. Praca obejmuje zagadnienia relacyjności baz danych, budowy warstwy komunikacyjnej backendu, implementacji logiki aplikacji pod kątem biznesowym oraz wdrożenie interfejsu frontendowego.

1.3 Cel pracy

Celem pracy jest zaprojektowanie i zaimplementowanie systemu webowego, który będzie pełnić funkcję inteligentnego kreatora zestawów komputerowych. System będzie dawać użytkownikowi możliwość wybrania dowolnych produktów z katalogu, sprawdzenie

czy aktualne komponenty znajdujące się na liście są wzajemnie kompatybilne, a gdy weryfikacja zakończy się pomyślnie, umożliwi realizację zamówienia na kompatybilny zestaw.

Idące za tym cele to:

- projekt i utworzenie relacyjnej bazy danych, odpowiadającej potrzebom sklepu internetowego,
- zaprojektowanie i implementacja backendu, odpowiedzialnego za komunikację i logikę aplikacji,
- wdrożenie przejrzystego frontendu, będącego interfejsem klienta,
- utworzenie grupy zasobów w środowisku chmurowym Azure (baza danych jako storage, aplikacja jako kontener).

1.4 Zakres prac

Zakres prac nad projektem obejmuje:

- projekt relacyjnej bazy danych w Oracle SQL Developer.
- utworzenie środowiska testowego serwera PostgreSQL, wprowadzenie gotowych danych o produktach do bazy danych,
- utworzenie testowych użytkowników oraz nadanie odpowiednich uprawnień każdemu z nich,
- implementację backendu w języku Python, implementacja API z wykorzystaniem frameworka FastAPI, umożliwiając integrację z bazą danych w celu zarządzania produktami i zamówieniami,
- projekt interfejsu użytkownika, stworzenie frontendu używając HTML, CSS oraz JavaScript,
- projekt kreatora zestawów komputerowych, zaimplementowanego z uwzględnieniem logik rozmytej, używając Reacta, Typescript, Node.js,
- migrację gotowej aplikacji z lokalnego serwera testowego PostgreSQL do produkcyjnej chmury Azure, utworzenie grupy zasobów.

1.5 Charakterystyka rozdziałów

Sekcja opisie każdy z rozdziałów pracy inżynierskiej, krótko charakteryzując przeznaczenie każdego z nich.

- **Rozdział 1** - zawiera wprowadzenie do tematyki pracy, przedstawia problem nieintuicyjnego doboru kompatybilnych podzespołów komputerowych, definiuje cel i zakres pracy oraz opisuje autorski wkład w projekt.
- **Rozdział 2** - obejmuje analizę tematu, w tym istotę kompatybilności sprzętowej, przegląd istniejących rozwiązań wykorzystywanych przez popularne sklepy komputerowe oraz porównanie ich funkcjonalności z zaprojektowanym systemem. Przybliża problematykę całej pracy.
- **Rozdział 3** - przedstawia wymagania funkcjonalne i нефunkcjonalne systemu, przypadki użycia, zastosowane narzędzia i technologie oraz metodykę pracy przyjętą podczas implementacji. Przedstawia kompletną listę dobranych technologii i rozwiązań.
- **Rozdział 4** - obejmuje wymagania niezbędne do uruchomienia aplikacji, przedstawia sposób uruchomienia systemu. Przedstawia również scenariusze korzystania z systemu z perspektywy użytkownika końcowego.
- **Rozdział 5** - stanowi część projektową, zawierającą architekturę systemu, opisanie każdego filaru aplikacji webowej, model danych wraz z diagramem ERD, opis struktury bazy danych, szczegółową analizę encji oraz omówienie kluczowych modułów, takich jak moduł weryfikacji kompatybilności i logiki rozmytej.
- **Rozdział 6** - prezentuje proces weryfikacji i walidacji systemu, zawierając opis sposobu testowania, organizacji eksperymentów, przypadków testowych oraz analizę wykrytych błędów i wyników testów. Analizuje skuteczność działania oprogramowania, porównując otrzymywane wyniki z założeniami.
- **Rozdział 7** - zawiera podsumowanie uzyskanych rezultatów, ocenę stopnia realizacji celów pracy oraz wskazuje możliwe kierunki dalszego rozwoju systemu. Podsumowuje i ocenia skuteczność podjętych działań, analizuje koszty poniesione podczas realizacji, szczegółowo ocenia dobór technologii.

1.6 Wkład autora

Na potrzeby realizacji projektu prace rozpoczęto od analizy problemu oraz zaplanowania funkcjonalności systemu, a następnie od doboru technologii odpowiednich do jego implementacji. Kolejnym etapem było zaprojektowanie relacyjnej bazy danych przy użyciu narzędzia Oracle SQL Developer Data Modeler, co umożliwiło utworzenie logicznego modelu danych oraz przygotowanie struktury tabel.

Na podstawie zaprojektowanego modelu utworzono testową instancję bazy danych PostgreSQL, wraz z konfiguracją użytkowników oraz nadaniem odpowiednich uprawnień dostępowych. Następnie, przy wykorzystaniu frameworka FastAPI, zaimplementowano warstwę backendową w języku Python, odpowiedzialną za komunikację z bazą danych oraz realizację logiki biznesowej systemu, w tym mechanizmów weryfikacji kompatybilności zestawów komputerowych.

Równolegle opracowano warstwę frontendową aplikacji, zapewniającą interaktywny i intuicyjny interfejs użytkownika. Zastosowane rozwiązania umożliwiają wygodne korzystanie z systemu zarówno przez użytkowników zaawansowanych, jak i osoby mniej doświadczone, wspierając proces doboru komponentów m.in. poprzez zastosowanie elementów logiki rozmytej.

W ramach akademickiej subskrypcji platformy Microsoft Azure zrealizowano proces hostingu aplikacji, jej administracji oraz testowania, obejmujący m.in. zagadnienia związane z konteneryzacją, wirtualizacją oraz automatyzacją procesów wdrożeniowych. Dzięki temu system został przygotowany do działania w środowisku zbliżonym do produkcyjnego oraz udostępniony w publicznej sieci internetowej.

Całość dokumentacji projektowej, w tym niniejsza praca inżynierska oraz opisy techniczne systemu, została opracowana samodzielnie w ramach realizacji projektu inżynierskiego pt. „System wspomagający inteligentne składanie zestawów komputerowych”.

Rozdział 2

Analiza problemu oraz przegląd dostępnych rozwiązań

Celem niniejszego rozdziału jest analiza problemu będącego przedmiotem pracy oraz przegląd istniejących rozwiązań stosowanych w podobnych systemach. Przeprowadzono analizę funkcjonalną dostępnych narzędzi oraz wskazano ich ograniczenia, które stanowiły podstawę do zaprojektowania własnego rozwiązania.

2.1 Sformułowanie problemu

Istota kompatybilności

Na rynku znajduje się szeroka gama komponentów komputerowych, która nieustannie poszerza się o nowe produkty. Różnice pomiędzy poszczególnymi modelami dotyczą przykładowo wymiarów fizycznych, dostępnych złączy, technologii socketów, czy minimalnej mocy zasilacza. Odpowiedni dobór części jest kluczowy do realizacji poprawnie funkcjonującego stanowiska komputerowego, gdyż dowolna niekompatybilność może prowadzić do braku możliwości fizycznego montażu lub niewłaściwej i nieciągłej pracy urządzenia. Nawet minimalne niezgodności mogą być przyczyną poważnej awarii, która może przyczynić się do strat finansowych, spowodowanych utratą danych, przerwaniem trwałości usług lub całkowitym uszkodzeniem sprzętu.

Niska dostępność narzędzi do weryfikacji kompatybilności

Aktualnie na polskim rynku sklepów komputerowych brakuje narzędzi umożliwiających użytkownikowi samodzielną i automatyczną weryfikację kompatybilności wybranych podzespołów. Większość sprzedawców oferuje wyłącznie gotowe zestawy komputerowe, które zapewniają pełną funkcjonalność, jednak często nie umożliwiają ich łatwej rozbudowy ani wymiany poszczególnych elementów na nowsze modele. W efekcie zaprojektowanie stanowiska w pełni odpowiadającego potrzebom użytkownika staje się utrudnione

i może wymagać konsultacji ze specjalistą, co generuje dodatkowe koszty oraz wydłuża czas realizacji zamówienia.

2.2 Analiza istniejących rozwiązań w najpopularniejszych sklepach na polskim rynku

Spośród wielu istniejących rozwiązań dostępnych na polskim rynku wybrano dwa, których poziom zaawansowania jest najwyższy i nie odbiega zbytnio od tego przyjętego przez autora.

Morele.net

Spośród przeanalizowanych platform sprzedażowych polskich sprzedawców, pod względem technicznym najlepiej zaprojektowanym rozwiązaniem okazał się konfigurator zestawów PC dostępny w sklepie Morele[7]. Narzędzie oferuje intuicyjny interfejs oraz możliwość wyboru szerokiej gamy podzespołów. Kluczowym ograniczeniem jest jednak brak automatycznej weryfikacji kompatybilności, gdzie w przypadku błędnie skonfigurowanego zestawu jedynie pracownik sklepu może ewentualnie skontaktować się z użytkownikiem. W praktyce znacząco opóźnia to proces realizacji zamówienia i niesie ryzyko zakupu podzespołów, które nie będą ze sobą wzajemnie kompatybilne. Dodatkowo konfigurator nie wykorzystuje logiki rozmytej przy sugerowaniu komponentów.

X-kom.pl

Drugim pod względem funkcjonalności rozwiązaniem jest konfigurator oraz oferta gotowych zestawów sklepu X-kom[22]. Platforma udostępnia czytelny interfejs oraz rekomendowane zestawy komputerowe, które zazwyczaj umożliwiają późniejszą rozbudowę. W porównaniu do Morele[7], użytkownik ma tutaj jeszcze mniejszą możliwość samodzielnego projektowania konfiguracji, ponieważ wybór ogranicza się do kilku lub kilkunastu proponowanych zestawów. Brak szczegółowej analizy potrzeb użytkownika oraz ograniczona liczba wariantów utrudniają złożenie w pełni dostosowanego zamówienia. Podobnie jak w przypadku Morele, sklep nie wykorzystuje logiki rozmytej w mechanizmach komponowania zestawów.

2.3 Porównanie istniejących rozwiązań z rozwiązaniem inżynierskim

Pierwszą i najważniejszą funkcjonalnością wyróżniającą opracowane rozwiązanie jest zastosowanie rozszerzonych mechanizmów weryfikacji kompatybilności podzespołów, któ-

rych brakuje w analizowanych sklepach internetowych. System wykorzystuje nie tylko klasyczne metody porównywania parametrów technicznych, lecz również elementy logiki rozmytej pozwalającej na bardziej elastyczną ocenę dopasowania podzespołów do oczekiwań użytkownika.

Aplikacja dokonuje szczegółowej analizy technicznej wszystkich wybranych komponentów, co przekłada się na sprawdzanie i weryfikowanie:

- zgodności procesora i płyty głównej pod względem zastosowanego socket'u,
- generację pamięci RAM oraz jej zgodności ze standardem płyty głównej,
- maksymalnej obsługiwanej częstotliwość i pojemność modułów pamięci RAM,
- parametrów energetycznych, takich jak zapotrzebowanie na moc oraz wydajności i maksymalnej dostarczanej mocy zasilacza,
- wymiarów kart graficznych oraz obudowy,
- zgodności chłodzenia procesora z wybranym gniazdem i limitem wysokości w obudowie.

W odróżnieniu od popularnych serwisów takich jak Morele.net[7] czy X-kom[22], które nie oferują pełnej automatycznej weryfikacji kompatybilności, system analizuje wszystkie istotne zależności pomiędzy podzespołami. Dzięki temu użytkownik nie jest narażony na ryzyko zakupu części wzajemnie niekompatybilnych, a proces konfiguracji zestawu przebiega szybciej, sprawniej i bez konieczności konsultacji ze specjalistą. Zastosowanie logiki rozmytej pozwala ponadto na uproszczenie docelowej kategoryzacji i oceny elementów dla użytkownika, bazując na dużej ilości danych wejściowych.

2.4 Logika rozmyta w systemach wspomaganie decyzji

Logika rozmyta stanowi rozszerzenie klasycznej logiki dwuwartościowej, w której każda zmienna może przyjmować jedynie wartości prawda lub fałsz. W odróżnieniu od podejścia klasycznego, logika rozmyta umożliwia modelowanie pojęć nieostrych oraz opis zjawisk, które nie mogą być jednoznacznie sklasyfikowane. Z tego względu znajduje ona szerokie zastosowanie w systemach wspomaganie decyzji, w szczególności w obszarach, gdzie występują kryteria jakościowe oraz subiektywne.

Podstawowym pojęciem logiki rozmytej jest zbiór rozmyty, w którym elementy należą do zbioru z określonym stopniem przynależności, przyjmującym wartości z przedziału od

0 do 1. Stopień ten określany jest za pomocą funkcji przynależności, która opisuje, w jakim stopniu dana wartość spełnia określone kryterium, na przykład „wysoka wydajność”, „niski pobór energii” lub „niska cena”.

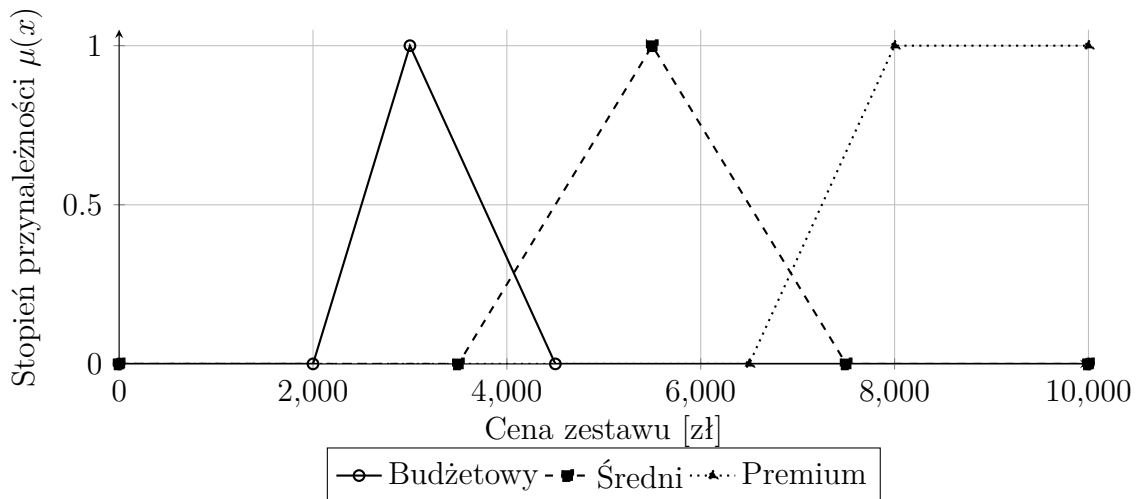
W kontekście systemów konfigurujących zestawy komputerowe logika rozmyta pozwala na bardziej elastyczną ocenę parametrów technicznych podzespołów, takich jak wydajność procesora, zapotrzebowanie energetyczne czy cena zestawu. Zamiast sztywnych progów decyzyjnych możliwe jest stopniowe przypisywanie konfiguracji do określonych kategorii, takich jak zestaw budżetowy, wydajny lub energooszczędny.

Proces wnioskowania w logice rozmytej składa się z kilku etapów:

- rozmywania (dopasowania wartości wejściowych do zbiorów rozmytych),
- zastosowania reguł rozmytych oraz wyostrażania, czyli przekształcenia wyniku rozmytego do postaci wartości jednoznacznej.

Dzięki temu możliwe jest uzyskanie końcowej oceny zestawu komputerowego w sposób zbliżony do ludzkiego procesu podejmowania decyzji.

Zastosowanie logiki rozmytej w niniejszym projekcie umożliwia klasyfikację zestawów komputerowych na podstawie wielu kryteriów jednocześnie, bez konieczności stosowania sztywnych warunków granicznych. Takie podejście zwiększa elastyczność systemu oraz poprawia jego użyteczność z perspektywy użytkownika końcowego. Na rysunku 2.1 przedstawiono przykładowe funkcje przynależności logiki rozmytej wykorzystywane do oceny kategorii cenowej zestawu komputerowego. Zastosowanie takich funkcji pozwala na płynne przejścia pomiędzy kategoriami oraz odwzorowanie nieostrych granic decyzyjnych.



Rysunek 2.1: Przykładowe funkcje przynależności dla kategorii cenowych zestawu komputerowego

Zastosowanie logiki rozmytej w niniejszym projekcie umożliwia pomocniczą, wielokryterialną ocenę zestawu komputerowego oraz jego przypisanie do określonej kategorii opi-

sowej. Uzyskana klasyfikacja stanowi dodatkową informację wspierającą analizę wybranej konfiguracji, bez narzucania jednoznacznych decyzji dotyczących doboru komponentów.

2.5 Architektura systemu teleinformatycznego aplikacji webowej

Współczesne systemy teleinformatyczne realizujące funkcje dostępne przez sieć Internet projektowane są często jako aplikacje webowe o architekturze klient-serwer. W takim modelu logicznym systemu warstwa kliencka, uruchamiana po stronie użytkownika końcowego (najczęściej w przeglądarce internetowej), odpowiada za prezentację danych oraz inicjowanie żądań, natomiast warstwa serwerowa realizuje przetwarzanie danych, logikę aplikacyjną oraz dostęp do zasobów trwałych. Komunikacja pomiędzy warstwami odbywa się z wykorzystaniem protokołu HTTP oraz jednoznacznie zdefiniowanych interfejsów programistycznych API.

Z punktu widzenia projektowania systemów teleinformatycznych istotną cechą tej architektury jest separacja funkcjonalna warstw systemu, polegająca na rozdzieleniu odpowiedzialności pomiędzy warstwę prezentacji, warstwę logiki aplikacyjnej oraz warstwę danych. Rozdzielenie to umożliwia niezależne projektowanie, implementację oraz testowanie poszczególnych komponentów systemu, a także ogranicza propagację zmian pomiędzy warstwami w trakcie dalszego rozwoju rozwiązania.

Warstwa kliencka systemu realizuje funkcje interakcji z użytkownikiem oraz wizualizacji wyników przetwarzania, natomiast serwer aplikacyjny odpowiada za obsługę żądań, walidację danych wejściowych, realizację przypadków użycia oraz komunikację z relacyjną bazą danych. Przyjęcie architektury bezstanowej (ang. *stateless*), charakterystycznej dla podejścia REST, oznacza brak przechowywania stanu sesji użytkownika po stronie serwera pomiędzy kolejnymi żądaniami. Stan konwersacji utrzymywany jest po stronie klienta lub przekazywany jawnie w ramach każdego żądania, co umożliwia równoległe przetwarzanie zapytań przez wiele instancji serwera.

2.6 Środowisko chmurowe jako środowisko wykonawcze systemu

Systemy teleinformatyczne realizujące usługi dostępne w trybie ciągłym projektowane są z założeniem ich uruchamiania w rozproszonym środowisku obliczeniowym. W przypadku aplikacji webowych rolę takiego środowiska pełnią platformy chmurowe, które dostarczają zasoby obliczeniowe, pamięciowe oraz sieciowe w postaci zwirtualizowanej infrastruktury. Środowisko chmurowe stanowi warstwę wykonawczą systemu, w której uruchamiane są komponenty aplikacyjne, usługi bazodanowe oraz elementy infrastruktury

sieciowej.

Z punktu widzenia architektury systemu teleinformatycznego istotnym aspektem jest separacja warstwy aplikacyjnej od warstwy infrastrukturalnej. Warstwa infrastrukturalna obejmuje zasoby obliczeniowe (np. maszyny wirtualne lub kontenery), system operacyjny, sieć wirtualną oraz mechanizmy kontroli dostępu. Warstwa aplikacyjna obejmuje uruchomione instancje serwera aplikacyjnego, usługi pośredniczące oraz bazę danych. Taki podział umożliwia niezależne zarządzanie konfiguracją infrastruktury oraz logiką aplikacyjną systemu.

Istotnym elementem środowiska chmurowego jest również zapewnienie ciągłości działania systemu w przypadku awarii pojedynczych komponentów. W architekturze rozproszonej awaria instancji aplikacji nie powoduje zatrzymania działania całego systemu, pod warunkiem istnienia innych aktywnych instancji zdolnych do obsługi ruchu. Mechanizmy monitorowania stanu usług oraz automatycznego ponownego uruchamiania komponentów stanowią integralną część środowiska wykonawczego systemu.

W kontekście bezpieczeństwa środowisko chmurowe wymusza jednoznaczne określenie granic odpowiedzialności pomiędzy aplikacją a infrastrukturą. Do zadań warstwy infrastrukturalnej należy m.in.:

- udostępnienie zwirtualizowanych zasobów obliczeniowych dla komponentów systemu,
- zapewnienie środowiska uruchomieniowego dla instancji serwera aplikacyjnego,
- realizacja mechanizmów routingu oraz przekazywania ruchu sieciowego do usług backendowych,
- równoważenie obciążenia pomiędzy wieloma instancjami komponentów serwerowych,
- izolacja zasobów infrastrukturalnych poszczególnych komponentów systemu,
- monitorowanie dostępności instancji oraz stanu zasobów infrastrukturalnych,
- automatyczne ponowne uruchamianie instancji w przypadku wykrycia awarii,
- zapewnienie ciągłości działania systemu przy częściowej niedostępności zasobów,
- zarządzanie konfiguracją środowiska uruchomieniowego aplikacji,
- kontrola dostępu do zasobów infrastrukturalnych,
- separacja środowisk uruchomieniowych (np. deweloperskiego i produkcyjnego),
- udostępnienie usług sieciowych niezbędnych do komunikacji komponentów systemu.

Przyjęcie środowiska chmurowego jako platformy wykonawczej stanowi spójny element architektury teleinformatycznej systemu.

2.6.1 Bezpieczeństwo systemu

Bezpieczeństwo systemu teleinformatycznego stanowi istotny aspekt jego projektowania i realizacji, w szczególności w przypadku aplikacji webowych przetwarzających dane użytkowników. W architekturze klient-serwerowej zagadnienia bezpieczeństwa obejmują zarówno ochronę transmisji danych pomiędzy komponentami systemu, jak i zabezpieczenie danych przechowywanych w warstwie serwerowej.

W niniejszym systemie przyjęto podejście polegające na oddzieleniu mechanizmów uwierzytelniania i autoryzacji od logiki biznesowej aplikacji. Dane uwierzytelniające użytkowników nie są przechowywane w postaci jawnej, lecz w formie skrótów kryptograficznych. Do tego celu stosowany jest algorytm bcrypt, który wykorzystuje mechanizm soli oraz parametryzowany koszt obliczeniowy, co ogranicza skuteczność ataków typu brute force oraz słownikowych w przypadku uzyskania nieautoryzowanego dostępu do bazy danych.

Rozdział 3

Wymagania i narzędzia

W rozdziale określone zostały wymagania funkcjonalne i нефункционалне projektowanego systemu oraz przedstawiono technologie i narzędzia wykorzystane podczas realizacji pracy. Zdefiniowane wymagania stanowią podstawę do dalszego projektowania architektury systemu.

3.1 Wymagania funkcjonalne

Wymagania funkcjonalne określają zachowania oraz funkcje, jakie system powinien realizować. Na podstawie analizy problemu wyodrębniono następujący zbiór wymagań:

- rejestracja oraz logowanie użytkowników,
- przeglądanie katalogu produktów sklepu komputerowego,
- dodawanie wybranych komponentów do kreatora zestawu PC,
- automatyczna weryfikacja kompatybilności podzespołów, obejmująca między innymi:
 - zgodność gniazda procesora,
 - typ pamięci RAM,
 - kompatybilność częstotliwości i pojemności pamięci RAM,
 - wymagania energetyczne,
 - dopasowanie komponentów do rozmiarów obudowy,
 - kompatybilność układów chłodzenia.
- wykorzystanie logiki rozmytej do klasyfikacji zestawów pod konkretne kategorie (np. budżetowy, wydajny, energooszczędny),
- dodawanie produktów oraz pełnych zestawów do koszyka,

- modyfikacja zawartości koszyka oraz finalizacja zamówienia.

3.2 Wymagania нефункционалне

Wymagania нефункционалне описују влаѣиwości jakościowe systemu, które wpływają na komfort użytkowania, bezpieczeństwo oraz niezawodność.

- **Skalowalność:** system powinien umożliwiać uruchomienie w środowisku chmurowym Microsoft Azure.
- **Безопасность:** hasła użytkowników muszą być przechowywane w formie zaszyfrowanej.
- **Независимость:** system powinien obsługiwać błędy kompatybilności w sposób kontrolowany i komunikatywny.
- **Удобность:** interfejs użytkownika powinien być przejrzysty, responsywny i intuicyjny.
- **Преносимость:** frontend oparty na technologii webowej dostosowanej do najpopularniejszych przeglądarek.
- **Высокая доступность:** system powinien być доступен в способ ciągły i несустанный.

3.3 Przykładowe scenariusze użycia systemu

Najważniejsze przypadki użycia systemu obejmują następujące scenariusze:

- „Ужыткownik rejestruje nowe konto”
- „Ужыткownik loguje się do systemu”
- „Ужыткownik przegląda dostępne komponenty”
- „Ужыткownik buduje własny zestaw PC”
- „System sprawdza kompatybilność elementów”
- „Ужыткownik dodaje zestaw do koszyka”
- „Ужыткownik realizuje zamówienie”

3.4 Narzędzia i technologie

Realizacja projektu wymagała zastosowania nowoczesnych technologii backendowych, frontendowych oraz narzędzi modelowania. Wszystkie zastosowane narzędzia i technologie przedstawiono w kolejnych podrozdziałach.

3.4.1 Frontend

- **React** [16] - biblioteka wykorzystywana do tworzenia interaktywnych interfejsów użytkownika.
- **TypeScript** [19] – rozszerzenie JavaScript umożliwiające statyczne typowanie.
- **Vite** [21] – bundler odpowiedzialny za budowanie i szybkie uruchamianie aplikacji.
- **Tailwind CSS** [18] – framework ułatwiający projektowanie responsywnego interfejsu.
- **Node.js** [8] – środowisko wykonawcze dla narzędzi JavaScript.
- **npm** [9] – menedżer pakietów odpowiedzialny za instalację zależności.
- **ESLint** [1] – narzędzie do analizy i kontroli jakości kodu TypeScript i JavaScript.

3.4.2 Backend

- **Python 3** [15] – język implementacji warstwy serwerowej.
- **FastAPI** [2] – framework umożliwiający tworzenie wydajnego REST API.
- **SQLAlchemy** [17] – ORM realizujący komunikację z bazą danych.
- **PostgreSQL** [12] – relacyjna baza danych przechowująca informacje o produktach i zamówieniach.
- **psycopg2** [13] – sterownik PostgreSQL wykorzystywany do komunikacji z bazą.
- **Pydantic** [14] – walidacja modeli danych wykorzystywana przez FastAPI.
- **Uvicorn** [20] – serwer ASGI obsługujący zapytania HTTP.

3.4.3 Modelowanie i narzędzia wspomagające

- **Oracle SQL Developer Data Modeler** [10] – narzędzie użyte do projektowania modelu ERD bazy danych.
- **Git** [3] – system kontroli wersji.
- **GitHub** [4] – repozytorium kodu oraz platforma do zarządzania wersjami projektu.
- **Microsoft Azure** [5] – środowisko do wdrożenia i hostowania bazy danych.

3.5 Metodyka pracy

Prace nad projektem prowadzono w sposób iteracyjny. Każdy cykl rozwoju obejmował:

- analizę oraz doprecyzowanie wymagań pod standardy rynkowe,
- projekt danej części funkcjonalnych systemu,
- implementację odpowiednio nowych modułów backendu i frontendu,
- testowanie, w przeznaczonym do tego środowisku, poprawności działania,
- refaktoryzację oraz optymalizację kodu.

Takie podejście umożliwiło równoległe rozwijanie bazy danych, logiki serwera i interfejsu użytkownika, a także szybkie reagowanie na pojawiające się problemy i dostosowywanie projektu do bieżących potrzeb oraz wyzwań.

Rozdział 4

Specyfikacja zewnętrzna

Celem niniejszego rozdziału jest przedstawienie specyfikacji zewnętrznej zaprojektowanego systemu. Rozdział opisuje wymagania niezbędne do uruchomienia aplikacji, sposób jej instalacji oraz obsługi, a także role użytkowników i aspekty związane z bezpieczeństwem. Przedstawiono również przykładowe scenariusze korzystania z systemu z perspektywy użytkownika końcowego.

4.1 Sposób instalacji systemu

System może zostać uruchomiony zarówno w środowisku lokalnym, jak i w środowisku chmurowym. W niniejszej sekcji przedstawiono sposób instalacji oraz uruchomienia aplikacji w obu wariantach.

4.1.1 Uruchomienie systemu w środowisku lokalnym

Uruchomienie systemu w środowisku lokalnym polega na lokalnym uruchomieniu warstwy frontendowej oraz backendowej aplikacji. W tym wariancie użytkownik końcowy korzysta z systemu za pośrednictwem przeglądarki internetowej, natomiast połączenie z bazą danych konfigurowane jest poprzez odpowiednie parametry w pliku konfiguracyjnym aplikacji.

W przypadku uruchomienia lokalnego wymagane jest posiadanie na komputerze zainstalowanego interpretera języka Python, środowiska Node.js oraz nowoczesnej przeglądarki internetowej. Warstwa frontendowa aplikacji budowana jest lokalnie przy użyciu skryptów środowiska Node.js, a następnie udostępniana jako aplikacja webowa dostępna pod adresem lokalnym `127.0.0.1`. Backend systemu uruchamiany jest jako serwer aplikacyjny w języku Python, który realizuje logikę biznesową oraz komunikację z bazą danych.

Domyślnie baza danych systemu hostowana jest w środowisku chmurowym, jednak w wariancie lokalnym możliwe jest również podłączenie do innej instancji bazy danych poprzez zmianę adresu IP oraz parametrów połączenia w pliku konfiguracyjnym aplikacji.

Takie rozwiązanie umożliwia elastyczne testowanie systemu bez konieczności utrzymywania lokalnej instancji bazy danych.

4.1.2 Uruchomienie systemu w środowisku chmurowym Azure

Alternatywnie system może zostać uruchomiony w całości w środowisku chmurowym Microsoft Azure. W tym wariantcie baza danych została wdrożona jako usługa Azure Database for PostgreSQL, natomiast warstwa backendowa aplikacji uruchamiana jest jako usługa serwerowa w ramach platformy Azure.

Proces wdrożenia obejmuje utworzenie odpowiednich zasobów chmurowych, takich jak instancja bazy danych oraz środowisko uruchomieniowe dla aplikacji backendowej. Po zakończeniu procesu wdrożenia system dostępny jest publicznie poprzez przeglądarkę internetową, bez konieczności instalacji dodatkowego oprogramowania po stronie użytkownika.

Takie podejście umożliwia uruchomienie aplikacji w środowisku zbliżonym do produkcyjnego oraz weryfikację poprawności jej działania w warunkach rzeczywistych.

4.2 Kategorie użytkowników

Zaprojektowany system przewiduje istnienie kilku kategorii użytkowników, różniących się zakresem dostępnych funkcjonalności. Podział na role użytkowników umożliwia logiczne rozgraniczenie uprawnień oraz odpowiednie zarządzanie dostępem do poszczególnych elementów systemu.

Role użytkowników zostały zdefiniowane w następujący sposób:

- Podstawową kategorią użytkowników są użytkownicy niezalogowani, którzy mają możliwość przeglądania katalogu dostępnych produktów oraz zapoznania się z funkcjonalnością systemu. Użytkownicy ci nie posiadają jednak możliwości zapisywania zestawów ani realizacji zamówień.
- Drugą kategorią są użytkownicy zarejestrowani. Po zalogowaniu uzyskują oni dostęp do rozszerzonej funkcjonalności systemu, obejmującej możliwość tworzenia i zapisywania zestawów komputerowych, dodawania produktów do koszyka oraz składania zamówień. Dane użytkowników są przechowywane w bazie danych systemu.

Zastosowany podział na kategorie użytkowników pozwala na zachowanie porządku w strukturze systemu oraz stanowi podstawę do dalszej rozbudowy aplikacji w przyszłości.

4.3 Sposób obsługi systemu

Obsługa systemu została zaprojektowana w sposób intuicyjny i niewymagający specjalistycznej wiedzy technicznej. Podstawowe etapy korzystania z systemu przedstawiono poniżej:

- Użytkownik uruchamia aplikację za pośrednictwem przeglądarki internetowej i uzyskuje dostęp do strony głównej systemu.
- System umożliwia przeglądanie katalogu dostępnych komponentów komputerowych oraz ich filtrowanie zgodnie z preferencjami użytkownika.
- Użytkownik przechodzi do kreatora zestawów komputerowych, w którym dodaje kolejne podzespoły do konfigurowanego zestawu.
- Podczas konfiguracji kreator na bieżąco analizuje zgodność wybieranych komponentów z wcześniej dodanymi elementami zestawu.
- W przypadku wykrycia niezgodności technicznej dalsza realizacja konfiguracji zostaje zablokowana, a użytkownik otrzymuje czytelny komunikat informujący o przyczynie problemu.
- Kreator uniemożliwia utworzenie zestawu niekompatybilnego technicznie, co eliminuje ryzyko popełnienia błędu przez użytkownika.
- Po skompletowaniu poprawnego zestawu użytkownik może zapisać konfigurację lub dodać ją do koszyka.
- Ostatnim etapem jest realizacja zamówienia, obejmująca podsumowanie wybranych komponentów oraz zatwierdzenie zamówienia.

4.4 Administracja systemem

Administracja systemem obejmuje czynności związane z zarządzaniem konfiguracją oraz danymi aplikacji. Funkcje administracyjne są ograniczone i przeznaczone do użytku przez właściciela systemu.

Zakres administracji systemem obejmuje w szczególności:

- zarządzanie katalogiem produktów oraz ich parametrami technicznymi przechowywanymi w bazie danych,
- nadzór nad poprawnością struktury danych oraz spójnością informacji wykorzystywanych przez mechanizm weryfikacji kompatybilności,

- aktualizację danych konfiguracyjnych systemu, w tym parametrów połączenia z bazą danych,
- kontrolę poprawności działania aplikacji backendowej oraz jej komunikacji z warstwą frontendową,
- wykonywanie czynności administracyjnych związanych z utrzymaniem systemu w środowisku lokalnym lub chmurowym.

4.5 Kwestie bezpieczeństwa

W trakcie projektowania systemu uwzględniono podstawowe aspekty bezpieczeństwa, mające na celu ochronę danych użytkowników oraz zapewnienie poprawnego i przewidywalnego działania aplikacji.

Najważniejsze zagadnienia związane z bezpieczeństwem systemu obejmują:

- przechowywanie danych użytkowników w relacyjnej bazie danych z zachowaniem integralności i spójności informacji,
- hasła użytkowników przechowywane są w bazie danych w postaci skrótów kryptograficznych wygenerowanych z wykorzystaniem algorytmu bcrypt, co uniemożliwia ich odtworzenie w postaci jawnej,
- walidację danych przekazywanych pomiędzy frontendem a backendem w celu ograniczenia ryzyka wprowadzenia niepoprawnych danych,
- ograniczenie dostępu do funkcji administracyjnych wyłącznie do roli administratora systemu,
- brak przechowywania wrażliwych danych po stronie klienta.

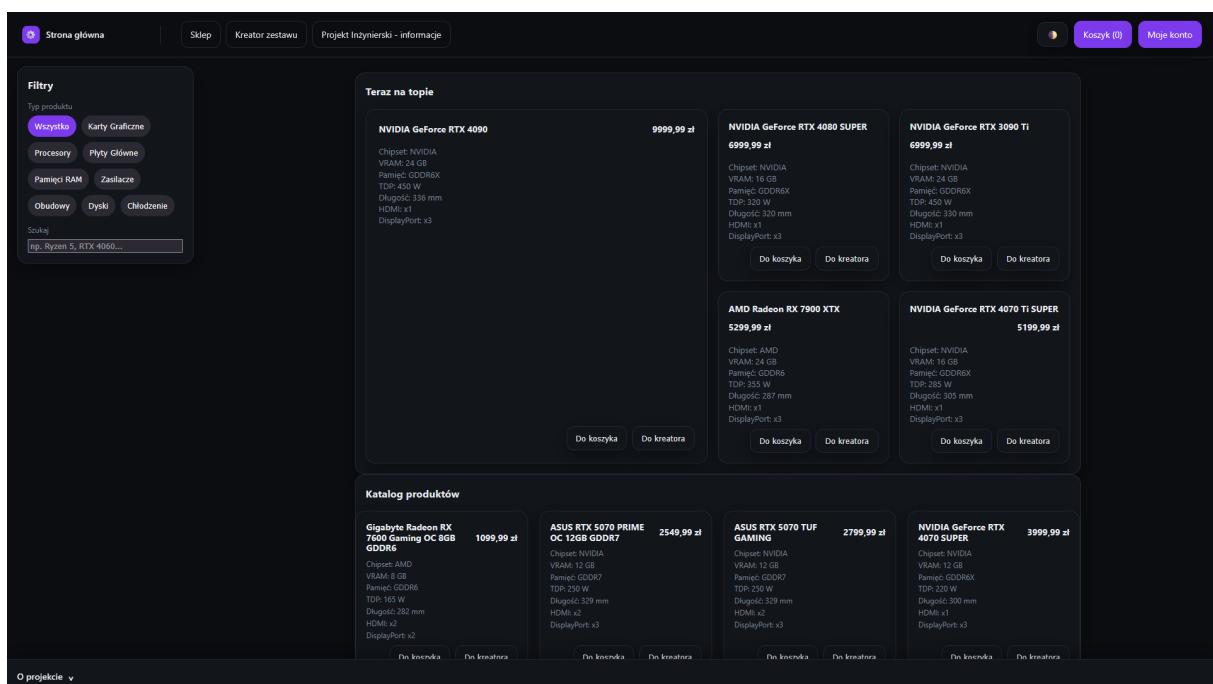
4.6 Scenariusze korzystania z systemu

W niniejszej sekcji przedstawiono przykładowe scenariusze korzystania z systemu z perspektywy użytkownika końcowego. Scenariusze ilustrują działanie kluczowych funkcjonalności aplikacji, w tym przeglądanie katalogu produktów, filtrowanie komponentów, zarządzanie koszykiem, proces logowania oraz działanie kreatora zestawów komputerowych. Każdy scenariusz został zilustrowany odpowiednimi zrzutami ekranu przedstawiającymi rzeczywiste działanie systemu.

4.6.1 Scenariusz 1: Przeglądanie katalogu komponentów

Użytkownik uruchamia aplikację i uzyskuje dostęp do strony głównej systemu, na której możliwe jest przeglądanie katalogu dostępnych komponentów komputerowych oraz zapoznanie się z ich podstawowymi parametrami technicznymi i ceną.

Na rysunku 4.1 przedstawiono widok strony głównej aplikacji wraz z listą dostępnych produktów.

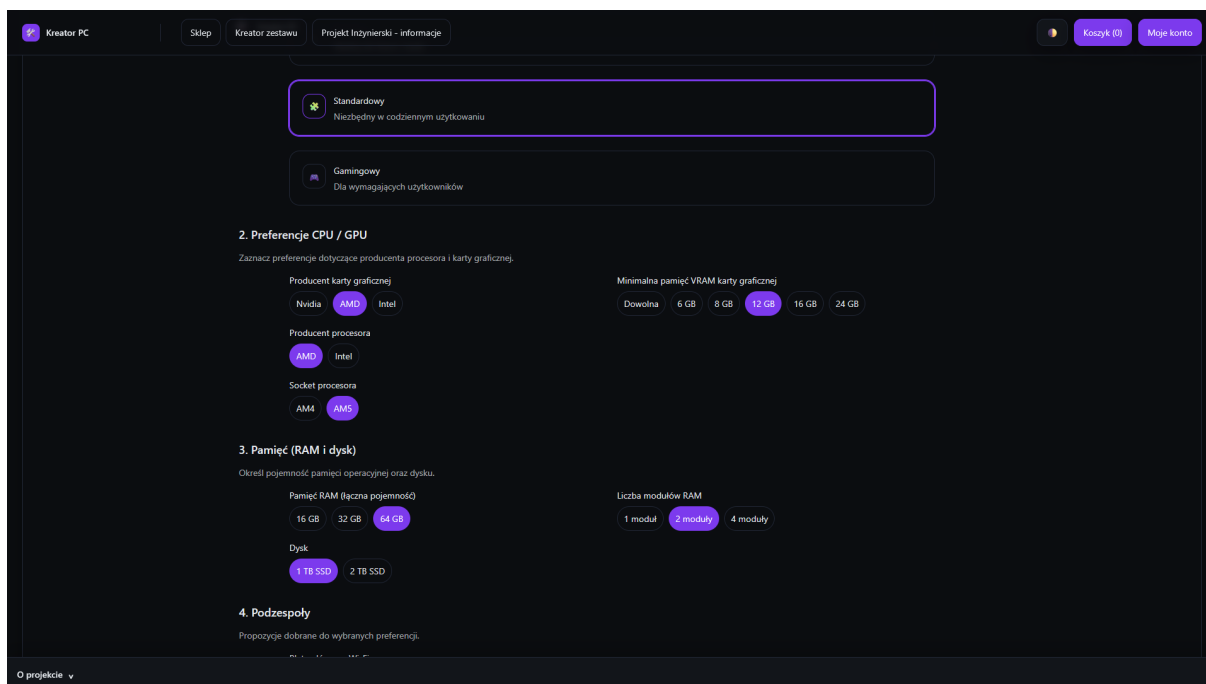


Rysunek 4.1: Widok strony głównej aplikacji

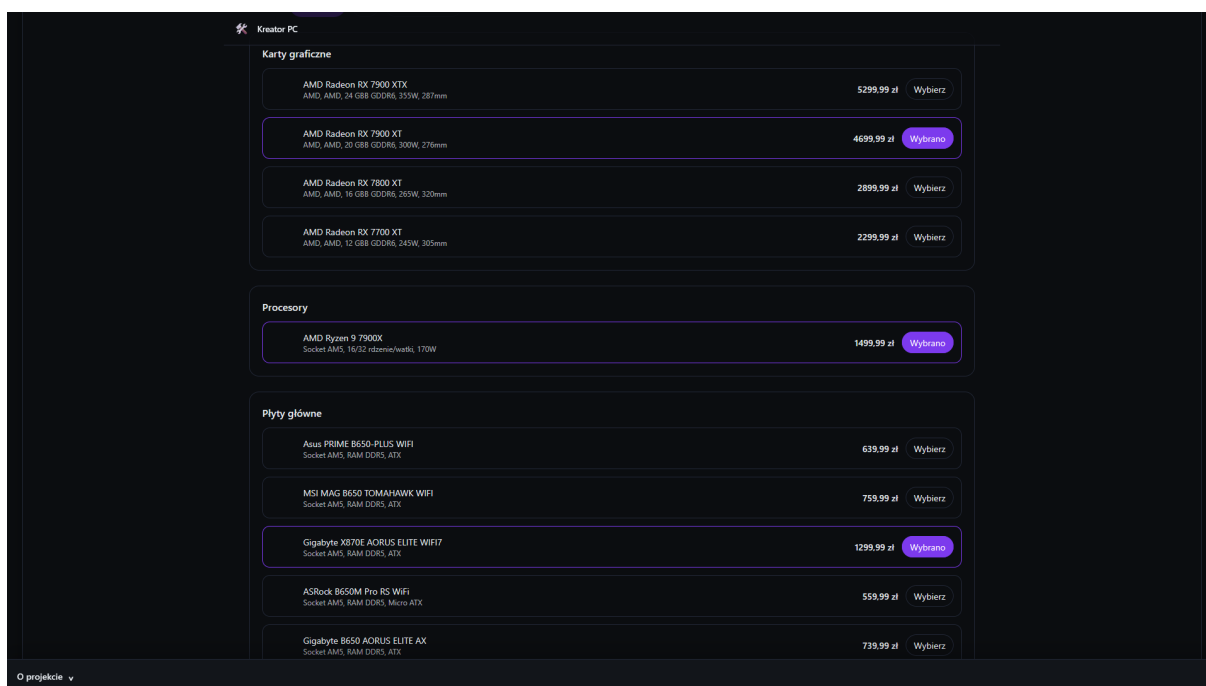
4.6.2 Scenariusz 2: Filtrowanie oraz wybór produktów

Użytkownik korzysta z dostępnych mechanizmów filtrowania produktów, takich jak typ komponentu, producent lub parametry techniczne, w celu zawężenia listy wyświetlanych elementów. Następnie wybiera interesujący go produkt z przefiltrowanej listy.

Proces filtrowania produktów przedstawiono na rysunku 4.2, natomiast wybór z komponentów z pośród przefiltrowanej listy na rysunku 4.3.



Rysunek 4.2: Filtrowanie produktów z katalogu

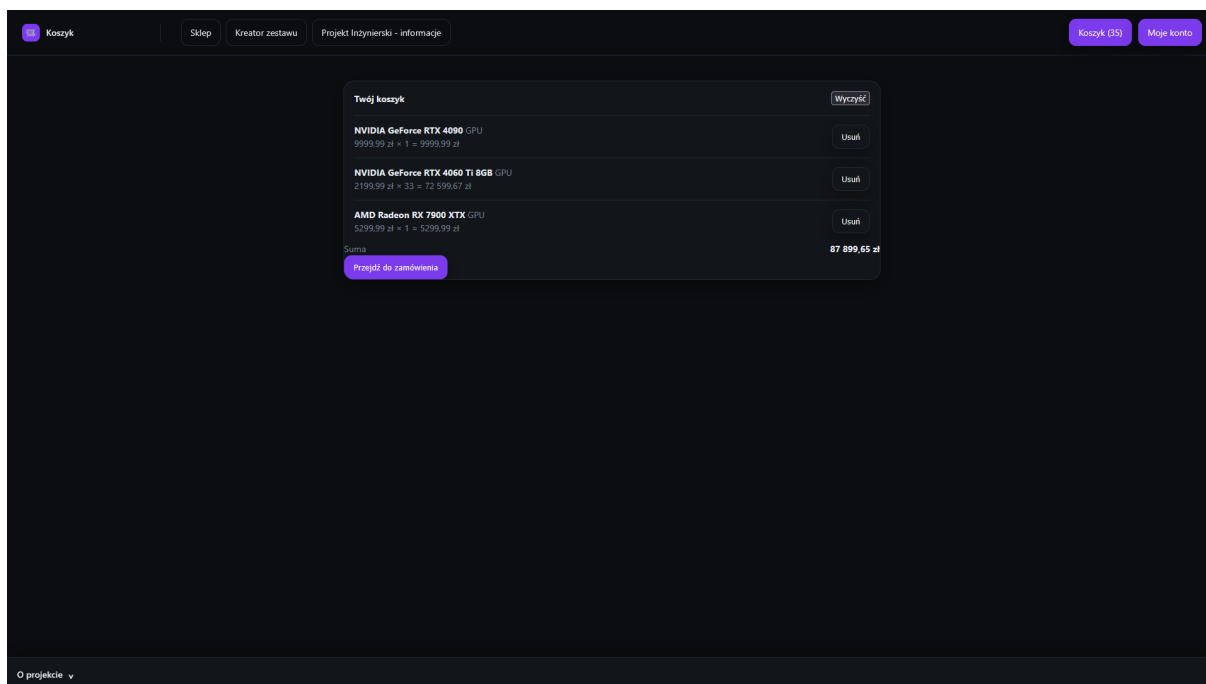


Rysunek 4.3: Wybór produktów z przefiltrowanego katalogu

4.6.3 Scenariusz 3: Dodawanie produktów do koszyka

Po wybraniu produktu użytkownik dodaje go do koszyka za pomocą przycisku dostępnego w widoku produktu. System zapisuje wybrany element w koszyku oraz wyświetla komunikat potwierdzający poprawne wykonanie operacji.

Na rysunku 5.8 przedstawiono widok koszyka zawierającego dodane produkty.

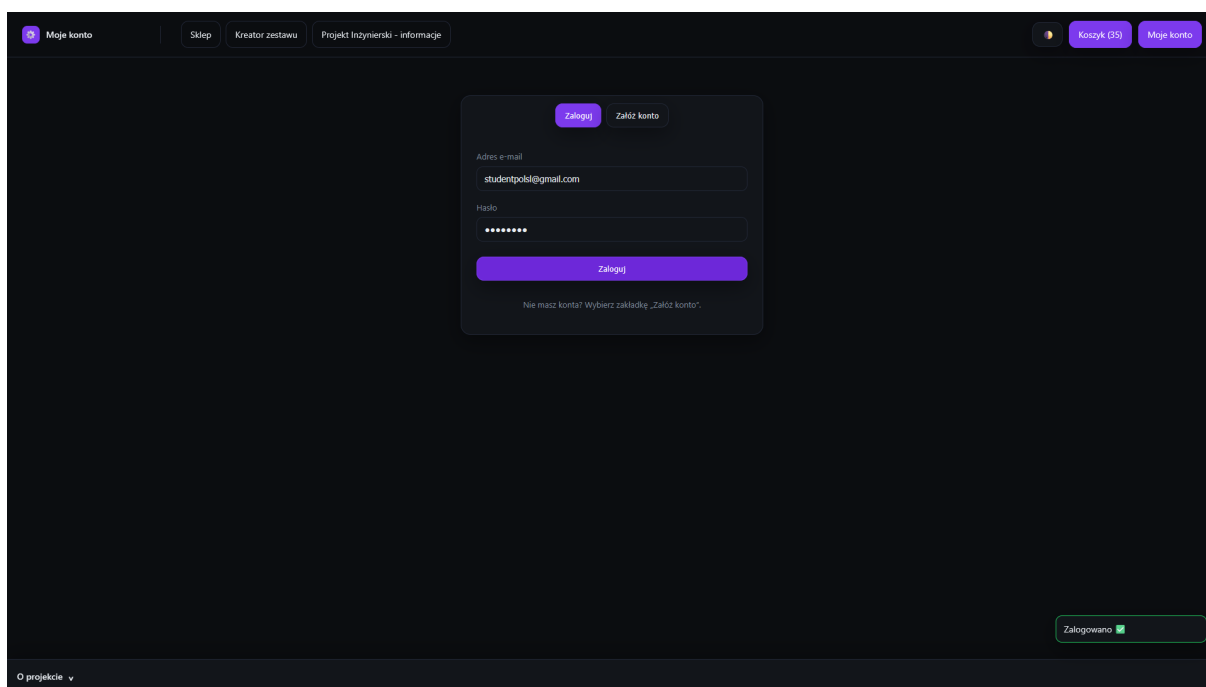


Rysunek 4.4: Widok koszyka użytkownika z dodanymi produktami

4.6.4 Scenariusz 4: Poprawne logowanie użytkownika

Użytkownik przechodzi do widoku logowania i wprowadza poprawne dane uwierzytelniające. Po ich weryfikacji system umożliwia dostęp do funkcji wymagających autoryzacji, takich jak zapis zestawu czy zarządzanie koszykiem.

Widok poprawnego procesu logowania przedstawiono na rysunku 4.5.

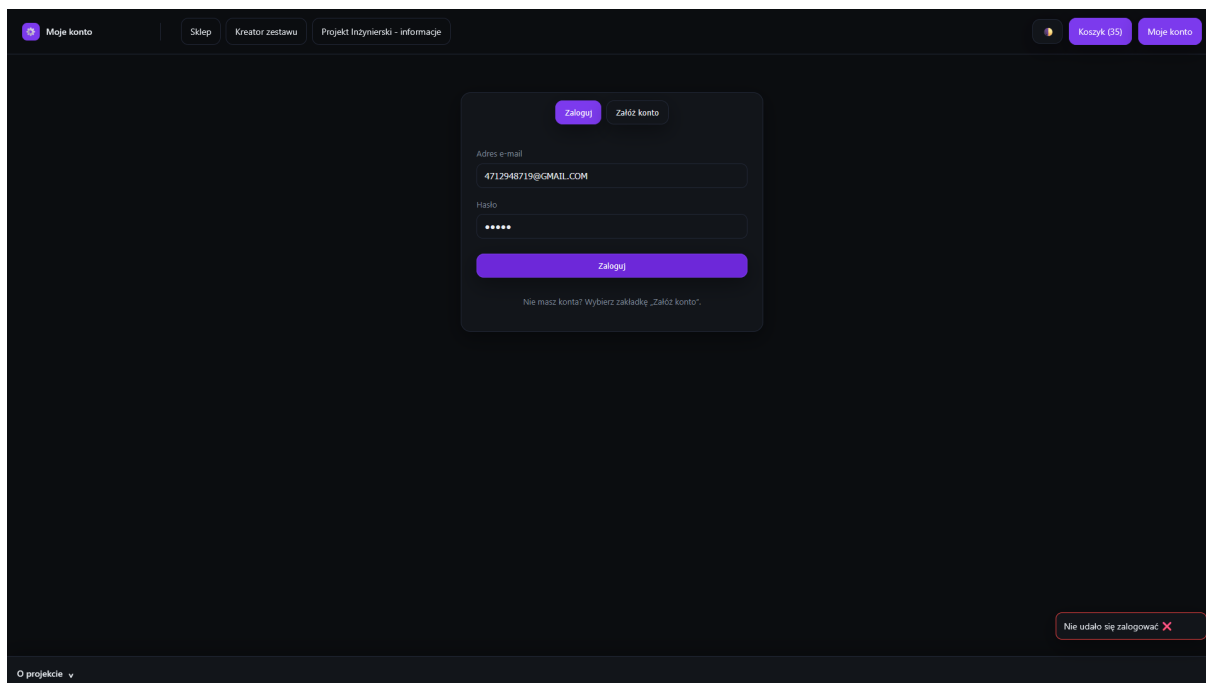


Rysunek 4.5: Poprawne logowanie użytkownika

4.6.5 Scenariusz 5: Niepoprawne logowanie użytkownika

W przypadku wprowadzenia niepoprawnych danych logowania system odrzuca próbę uwierzytelnienia i wyświetla stosowny komunikat informujący o błędzie.

Sytuację tę przedstawiono na rysunku 4.6.

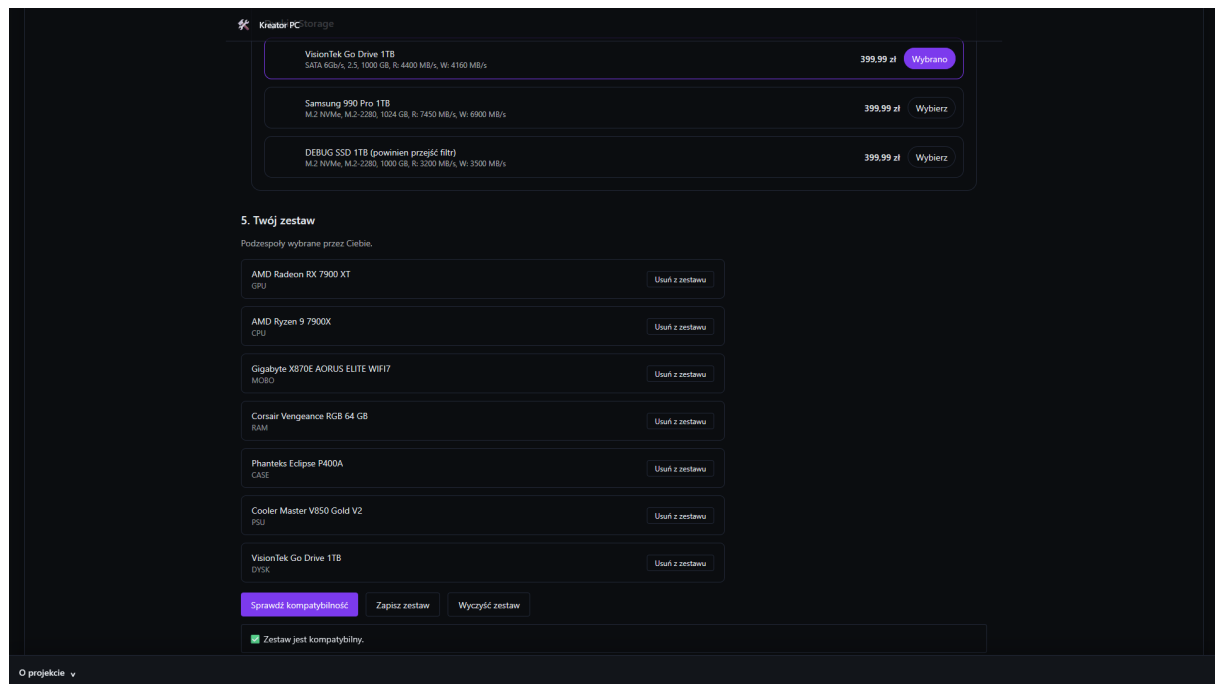


Rysunek 4.6: Komunikat o niepoprawnych danych logowania

4.6.6 Scenariusz 6: Utworzenie kompatybilnego zestawu komputerowego

Użytkownik przechodzi do kreatora zestawów komputerowych i wybiera kompatybilne podzespoły, takie jak procesor, płyta główna, pamięć RAM oraz pozostałe elementy konfiguracji. System na bieżąco weryfikuje zgodność techniczną wybieranych komponentów.

Po skompletowaniu poprawnego zestawu użytkownik może zapisać konfigurację lub dodać ją do koszyka. Widok poprawnie skonfigurowanego zestawu przedstawiono na rysunku 4.7.

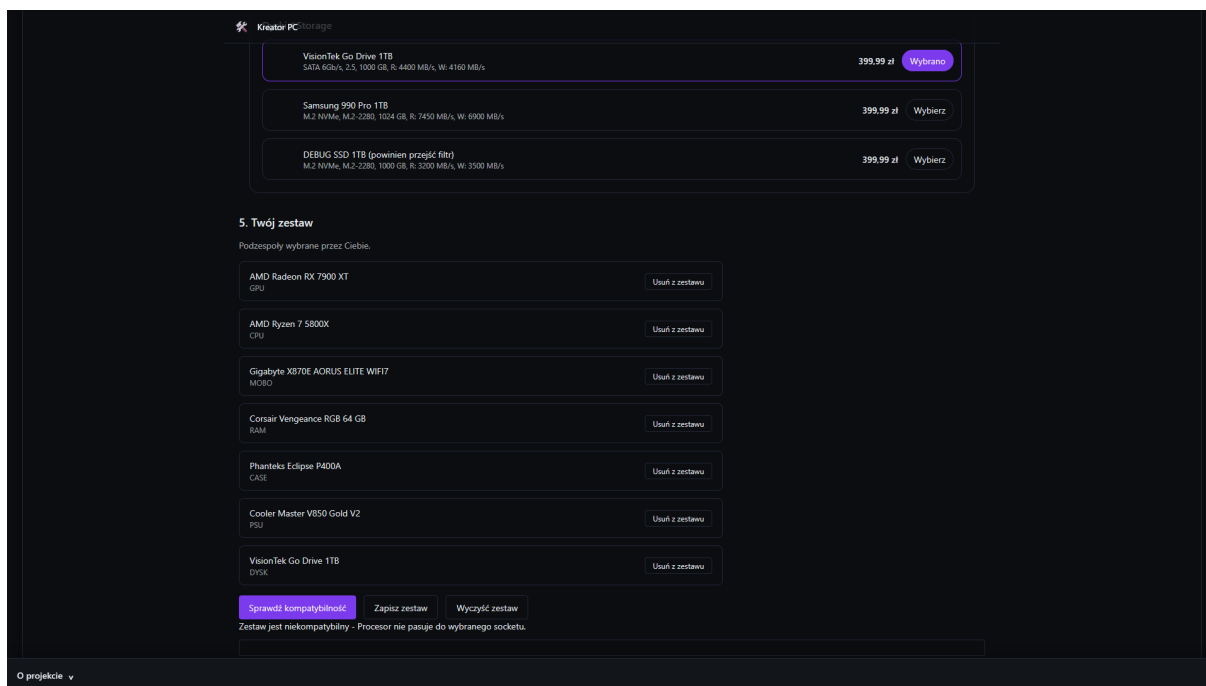


Rysunek 4.7: Widok kreatora zestawów komputerowych

4.6.7 Scenariusz 7: Próba utworzenia niekompatybilnego zestawu

W tym scenariuszu użytkownik próbuje skonfigurować zestaw komputerowy zawierający niezgodne technicznie podzespoły, na przykład procesor oraz płytę główną obsługujące różne typy gniazd.

System wykrywa niezgodność i blokuje możliwość zapisania konfiguracji, wyświetlając komunikat informujący o przyczynie problemu, co przedstawiono na rysunku 4.8.



Rysunek 4.8: Komunikat o niezgodności podzespołów

Rozdział 5

Specyfikacja wewnętrzna

Celem niniejszego rozdziału jest przedstawienie projektu oraz architektury zaprojektowanego systemu. Zaprezentowano w nim strukturę systemu, podział na warstwy, model danych, architekturę backendu i frontendu oraz zastosowane rozwiązania techniczne.

5.1 Przedstawienie idei systemu

System jest odpowiedzią na nieustannie poszerzający się rynek podzespołów komputerowych, pełniąc funkcję inteligentnego asystenta przy kompletowaniu osobistego zestawu komputerowego. Implementacja logiki rozmytej miała na celu ułatwienie nawet nieposiadającym specjalistycznej wiedzy użytkownikom skuteczne przejście przez ten proces, uwzględniając ich potrzeby takie jak wydajność, energooszczędność czy odpowiedni koszt. System łączy intuicyjny interfejs, znany z innych serwisów internetowych, z zaawansowanymi mechanizmami wspomagającymi, zaprojektowanymi konkretnie pod potrzeby klientów.

5.1.1 Cel powstania aplikacji

Celem systemu było uproszczenie samodzielnego składania zestawów komputerowych poprzez inteligentną asekurację podczas tego procesu. Jego celem jest eliminacja potencjalnych błędów po stronie użytkowników.

Aplikację projektowano z myślą o:

- intuicyjnym wyborze komponentów z katalogu,
- automatycznej weryfikacji kompatybilności,
- wsparciu w dopasowaniu sprzętu do potrzeb, używając logiki rozmytej,
- możliwości zapisaniu kompatybilnego zestawu oraz o jego zamówieniu.

5.1.2 Innowacyjność systemu

System powstał jako inteligentny kreator zestawów komputerowych, składający się z mechanizmów weryfikacji oraz wspomagania decyzji, z intencją udoskonalenia mechanizmów najpopularniejszych polskich sklepów komputerowych.

Innowacyjność systemu wynika z:

- automatycznej weryfikacji kompatybilności, obejmującej wszystkie powszechne znane parametry, takie jak:
 - socket procesora i płyty głównej,
 - typ pamięci RAM,
 - format obudowy,
 - maksymalna suma obciążenia dla konkretnego zasilania,
 - fizyczne wymiary elementów,
 - kompatybilność portów, gniazd oraz złącz.
- zastosowania logiki rozmytej, umożliwiającej dobranie odpowiedniego zestawu pod indywidualne potrzeby, dzieląc je na:
 - budżet,
 - energooszczędność,
 - przeznaczenie.
- zapewnionego nieustannego monitorowania stanu aplikacji, dbając o wysoką dostępność serwisu i jego wydajność,
- potencjału rozbudowywania aplikacji w rzeczywistym środowisku produkcyjnym, poprzez dbanie o skalowalność i integralność z najnowszymi technologiami.

5.2 Architektura systemu

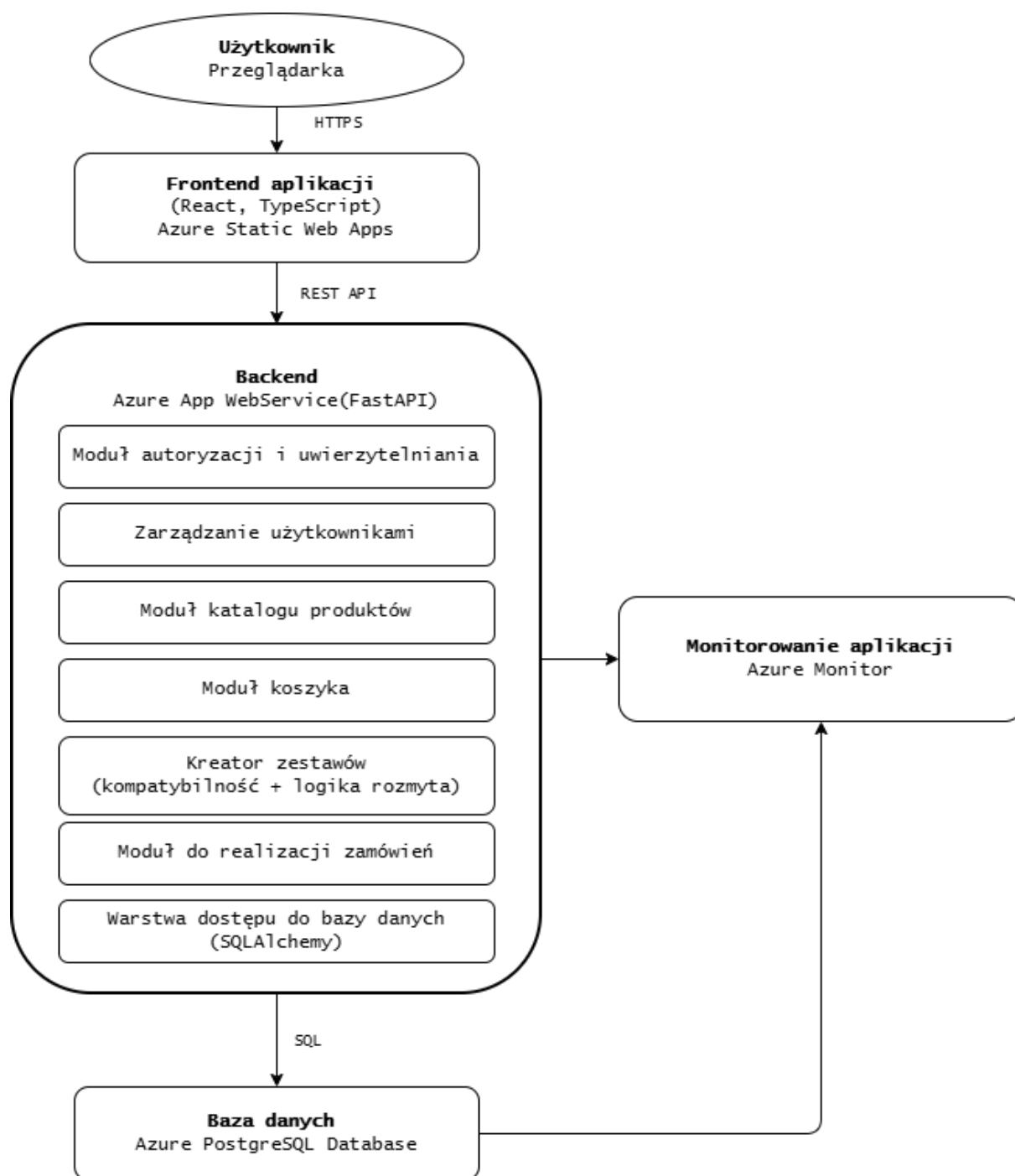
Sekcja ta skupia się na szczegółowej analizie architektury systemu teleinformatycznego, przedstawiając schemat ideowy działania aplikacji oraz omawiając sposób współpracy poszczególnych komponentów systemu.

5.2.1 Schemat ideowy

Na rysunku 5.1 przedstawiono schemat ideowy działania zaprojektowanego systemu. Diagram obrazuje podział aplikacji na warstwę frontendową, backendową oraz warstwę bazy danych, a także sposób komunikacji pomiędzy tymi elementami.

Warstwa frontendowa odpowiedzialna jest za interakcję z użytkownikiem oraz prezentację danych i została szczegółowo opisana w rozdziale 3. Warstwa backendowa realizuje logikę biznesową systemu, w tym mechanizmy weryfikacji kompatybilności podzespołów, obsługę koszyka oraz proces uwierzytelniania użytkowników, co omówiono w rozdziale 5. Struktura oraz projekt bazy danych, stanowiącej centralny element przechowywania informacji, zostały przedstawione w rozdziale 4.

Schemat ideowy pozwala na całościowe spojrzenie na architekturę systemu oraz ułatwia zrozumienie zależności pomiędzy poszczególnymi modułami aplikacji, które zostały szczegółowo omówione w kolejnych częściach pracy.



Rysunek 5.1: Schemat ideowy działania aplikacji

5.2.2 Frontend

Frontend aplikacji pełni ważną funkcję w funkcjonowaniu każdego systemu, ponieważ odpowiada za dostęp użytkownika do funkcjonalności oferowanych przez backend. Projekt wykorzystuje nowoczesne technologie React, TypeScript oraz Vite, zapewniając innowacyjność, wydajność oraz skalowalność. Inteligentny kreator zestawów komputerowych to jednostronicowy typ aplikacji webowej, dzięki czemu przełączanie widoku następuje dynamicznie bez konieczności ponownego ładowania zasobów. Takie rozwiązanie gwarantuje wysoką wydajność, prostotę oraz komfort użytkowania.

Zadania realizowane przez interfejs:

- prezentacja katalogu produktów, pobieranych dynamicznie z backendu,
- wybranie komponentów do przeniesienia z katalogu do kreatora,
- obsługa koszyka, dodawanie, modyfikowanie i usuwanie elementów,
- reakcja na działania użytkownika wykorzystując React Hooks,
- wyświetlanie aktualnego stanu kompatybilności zestawów,
- wykonywanie zapytań REST API do backendu w celu weryfikacji danych i aktualizacji stanu aplikacji.

Za wygląd zgodny ze standardami rynkowymi odpowiedzialny jest Tailwind CSS, gwarantując responwność i estetykę rozwiązania. Framework umożliwia konfigurację i nadawanie stylów bezpośrednio w komponentach poprzez gotowe klasy utility. Takie rozwiązanie zapewnia intuicyjne i dynamiczne UI. Za komunikację z backendem odpowiada REST API poprzez zapytania realizowane biblioteką fetch API. Każda interakcja ze stroną powoduje wywołanie właściwego endpoint'u, co dynamicznie aktualizuje stan aplikacji, w tym bazę danych. Projekt wdrożony został w chmurze Microsoft Azure, jako Azure Static Web App, pełniącej rolę hostingu. Całość stacku technologicznego zapewnia wysoką dostępność aplikacji oraz łatwość korzystania i zarządzania.

Projekt frontendu powstał z myślą o:

- przejrzystości interfejsu,
- prostocie korzystania,
- szybkości działania.

Takie podejście powoduje, że aplikacja jest łatwa w utrzymaniu, skalowalna, wysoko dostępna oraz przejrzysta zarówno dla początkujących jak i zaawansowanych użytkowników.

5.2.3 Backend

Backend odpowiada za logikę projektu, przetwarzanie danych oraz za kluczową komunikację kodu z bazą danych. Implementacja w języku Python, z wykorzystaniem frameworka FastAPI zapewnia nowoczesność i wydajność usług sieciowych, gwarantując przejrzystość, skalowalność oraz integralność z frontendem. Backend pełni funkcję pośrednika pomiędzy interfejsem użytkownika a bazą danych, gdzie żądania od strony użytkownika są nieustannie przetwarzane, a następnie przekazywane do odpowiednich modułów. Zwracana odpowiedź, w formacie JSON, umożliwia płynną komunikację w architekturze aplikacji.

Struktura omawianej warstwy zawiera następujące moduły funkcjonalne:

- moduł autoryzacji i uwierzytelniania - odpowiada za rejestrację, logowanie i odpowiednie weryfikowanie danych użytkownika. Wykorzystuje algorytm bcrypt do zapewnienia bezpieczeństwa haseł,
- moduł zarządzania użytkownikami - odpowiada za rejestrację i logowanie, co bezpośrednio umożliwia monitorowanie historii zamówień, poprzez tworzenie identyfikatora każdego klienta,
- moduł katalogu produktów - umożliwia przeglądanie oferty sklepu, pobieranie listy części, filtrowanie i sortowanie,
- moduł kreatora - odpowiada za dobór wzajemnie kompatybilnych komponentów, umożliwia utworzenie listy części komputerowych,
- moduł weryfikacji kompatybilności - odpowiada za porównywanie parametrów elementów zestawu, analizując zgodność pomiędzy nimi, stanowiąc rdzeń i główną funkcjonalność projektu,
- moduł logiki rozmytej - odpowiedzialny jest za klasyfikację zestawu do jednej z kategorii,
- moduł koszyka i zamówień - zadaniem tego modułu jest dodawanie elementów do koszyka, nieustanne monitorowanie jego stanu i modyfikowanie zawartości.

Komunikacja pomiędzy backendem a bazą danych zaprojektowana została przy pomocy SQLAlchemy, który pełni funkcję warstwy między kodem aplikacji, a relacyjną bazą danych (warstwa ORM). Takie rozwiązanie zapewnia odwzorowanie struktury tabel w postaci obiektów, znacząco ułatwiając implementację logiki, minimalizując ryzyko błędów oraz zwiększając bezpieczeństwo. Operacje wykonywane w ten sposób określamy jako bezpieczne i atomowe. Wdrożenie w postaci Azure App Service, zapewnia skalowalność, prostotę utrzymania oraz wysoką dostępność, co jest niezwykle istotne podczas projektowania aplikacji przeznaczonej dla klientów.

Backend zaprojektowany został z myślą o:

- wysokiej wydajności,
- przejrzystości kodu,
- bezpieczeństwie danych,
- skalowalności,
- niezawodności,
- wysokiej dostępności.

Realizacja tej części projektu w sposób opisany powyżej stanowi stabilną i skalowalną podstawę działania aplikacji, gwarantując poprawne komunikowanie się żądań użytkownika z bazą danych.

5.2.4 Baza danych

Baza danych jest fundamentem każdej aplikacji biznesowej, gdyż odpowiedzialna jest za trwałe przechowywanie informacji o użytkownikach, produktach oraz zamówieniach. Zaprojektowana została w oparciu o zasady normalizacji oraz analizę charakterystycznych właściwości komponentów, zapewniając integralność danych oraz efektywne weryfikowanie kompatybilności elementów.

Model i schemat danych zaprojektowany został przy użyciu Oracle SQL Developer Data Modeler, co umożliwiło utworzenie diagramu ERD oraz automatyczne wygenerowanie definicji tabel. Wyeksportowana baza jako plik DDL wdrożona została do środowiska DataGrip, w którym wypełniono ją przygotowanymi danymi części komputerowych, pochodzącymi z publicznego repozitorium komponentów[11], w celu implementacji danych produktów. System operował w oparciu o PostgreSQL, pełniący funkcję środowiska testowego wykorzystywanego do sprawnego implementowania backendu i modułów logiki. Gdy projekt przeszedł do etapu wdrożenia produkcyjnego, baza danych została zmigrowana do chmury Microsoft Azure, korzystając z gotowych poleceń podanych w dokumentacji Microsoft Azure[6]. Przeniesiona struktura zaimplementowana została jako Azure PostgreSQL Database, co dzięki funkcjonalności grupowania zasobów Azure zapewnia integrację z produkcyjną wersją backendu uruchomioną w Azure App Service.

Baza Danych spełnia wymagi trzeciej postaci normalnej (3NF), co oznacza, że nie zawiera redundancji danych, wszystkie atrybuty niekluczowe są w pełni zależne od klucza głównego właściwych tabel. Właściwość ta pozwala na:

- zminimalizowanie ryzyka niespójnych danych,
- zwiększenie wydajności zapytań,
- eliminację duplikatów.

Kluczową rolę w bazie danych pełni tabela Produkty, będąc ogólnym rejestrem wszystkich komponentów dostępnych w ofercie. Przechowuje ona podstawowe informacje o produktach, natomiast szczegółowe parametry znajdują się w dedykowanych tabelach zależnych od typu podzespołu. Takie rozwiązanie eliminuje redundancję danych oraz dodaje przejrzystości do przechowywania właściwości.

W celu przechowywania wielu komponentów w jednym zestawie, koszyku lub zamówieniu zastosowano liczne relacje typu M:N, które zostały zrealizowane przy pomocy tabel łączących. Całość struktury została zabezpieczona kluczami obcymi, co zapewnia pełną integralność referencyjną oraz uniemożliwia tworzenie błędnych powiązań między danymi. Modularny charakter modelu umożliwia jego dalszy rozwój, w tym dodawanie nowych typów komponentów.

Dzięki zastosowaniu odpowiednich narzędzi, normalizacji oraz chmurowej infrastruktury Azure, baza danych stanowi stabilny i skalowalny fundament systemu, wspierający zarówno logikę kompatybilności, jak i zarządzanie koszykami, zamówieniami oraz zestawami użytkowników.

5.2.5 Infrastruktura chmurowa

Wybór infrastruktury chmurowej jest kluczowym aspektem każdego projektu nastawionego na wysoką dostępność. Platforma Microsoft Azure gwarantuje skalowalność i bezpieczne środowisko, umożliwiając integrację z poszczególnymi komponentami systemu oraz automatyzację wdrażania kolejnych części aplikacji. Organizację pracy znacząco upraszcza ulokowanie środowiska produkcyjnego w ramach grupy zasobów Azure, która zbiera całość elementów w jednej strukturze administracyjnej. Takie rozwiązanie gwarantuje intuicyjne i kontrolowane zarządzanie usługami, co pozwala na ciągłe monitorowanie zużycia zasobów oraz kosztów utrzymania serwisu.

Projekt został uruchomiony z wykorzystaniem kredytu Azure o równowartości 100 USD, przyznanego przez Microsoft w ramach programu Azure for Students. Umożliwiło to wdrożenie aplikacji bez generowania dodatkowych kosztów w początkowym okresie. Przyznane środki pozwoliły na uruchomienie wszystkich wymaganych usług, takich jak:

- Azure Static Web App – hosting warstwy frontendu,
- Azure App Service – hosting warstwy backendu,
- Azure Database for PostgreSQL – baza danych dla aplikacji.

Azure Static Web App

Warstwa frontendu została zaimplementowana w usłudze Azure Static Web Apps, co umożliwiło udostępnienie strony internetowej użytkownikom. Usługa ta obsługuje natywną integrację z warstwą backend.

Jest to lekkie i skalowalne rozwiązanie, zapewniające obsługę wielu użytkowników jednocześnie. Dzięki wykorzystaniu Azure Static Web App wdrożenie frontendu było proste, a publikacja zmian – zautomatyzowana.

Azure App Service

Warstwa backendu została wdrożona z użyciem usługi Azure App Service. Aplikacja oparta na frameworku FastAPI (w języku Python) została uruchomiona na platformie zarządzanej przez Azure, co zapewniło:

- uproszczone wdrażanie aktualizacji,
- łatwe zarządzanie środowiskiem wykonawczym,
- integrację z usługami monitorującymi Azure,
- stabilność działania i wysoką dostępność

Takie rozwiązanie zwiększyło odporność backendu na duże obciążenia i znacząco zredukowało konieczność ręcznej administracji w czasie działania systemu (większość zadań infrastrukturalnych jest automatyzowana przez platformę Azure).

Azure Database for PostgreSQL

Dane aplikacji przechowywane są w zarządzanej bazie danych Azure Database for PostgreSQL, która zapewnia środowisko do wygodnego zarządzania relacyjną bazą danych. Wdrożenie bazy danych polegało na migracji istniejącej bazy PostgreSQL do platformy Azure, co obejmowało:

- eksport struktury i danych ze środowiska lokalnego,
- dostosowanie typów danych do wymogów Azure,
- weryfikację integralności po migracji.

Takie rozwiązanie gwarantuje wysoką niezawodność oraz elastyczny dostęp do danych z dowolnego miejsca. Przechowywanie danych w chmurze umożliwia skalowanie i dostęp do bazy w zależności od potrzeb, jednocześnie redukując koszty oraz obciążenia związane z utrzymywaniem własnej infrastruktury serwerowej.

Azure Monitor

Aplikacja była monitorowana w czasie rzeczywistym za pomocą usług Azure Monitor oraz Application Insights. Pozwoliło to na bieżące wykrywanie ewentualnych problemów, reagowanie na zwiększony ruch oraz automatyczne dopasowywanie zasobów do aktualnych potrzeb. Dzięki tym narzędziom zespół mógł szybko identyfikować wąskie gardła i zapewnić stabilne działanie systemu w trakcie trwania projektu.

Koszty i optymalizacja

Na etapie wdrażania przeanalizowano potencjalne koszty utrzymania aplikacji i dobrano rozwiązanie o możliwie najniższych parametrach, odpowiednie do spodziewanego niewielkiego ruchu. Wprowadzone ograniczenia nie wpłynęły negatywnie na funkcjonalność systemu, a pozwoliły zaoszczędzić środki – wykorzystanie nawet średniej klasy komponentów mogłoby przekroczyć założony budżet. Zastosowano następujące działania optymalizacyjne:

- wybór najniższych dostępnych planów usług (o minimalnych parametrach wydajności) dla hostingu,
- włączenie automatycznego wstrzymywania działania aplikacji przy minimalnym ruchu (poniżej zdefiniowanego progu),

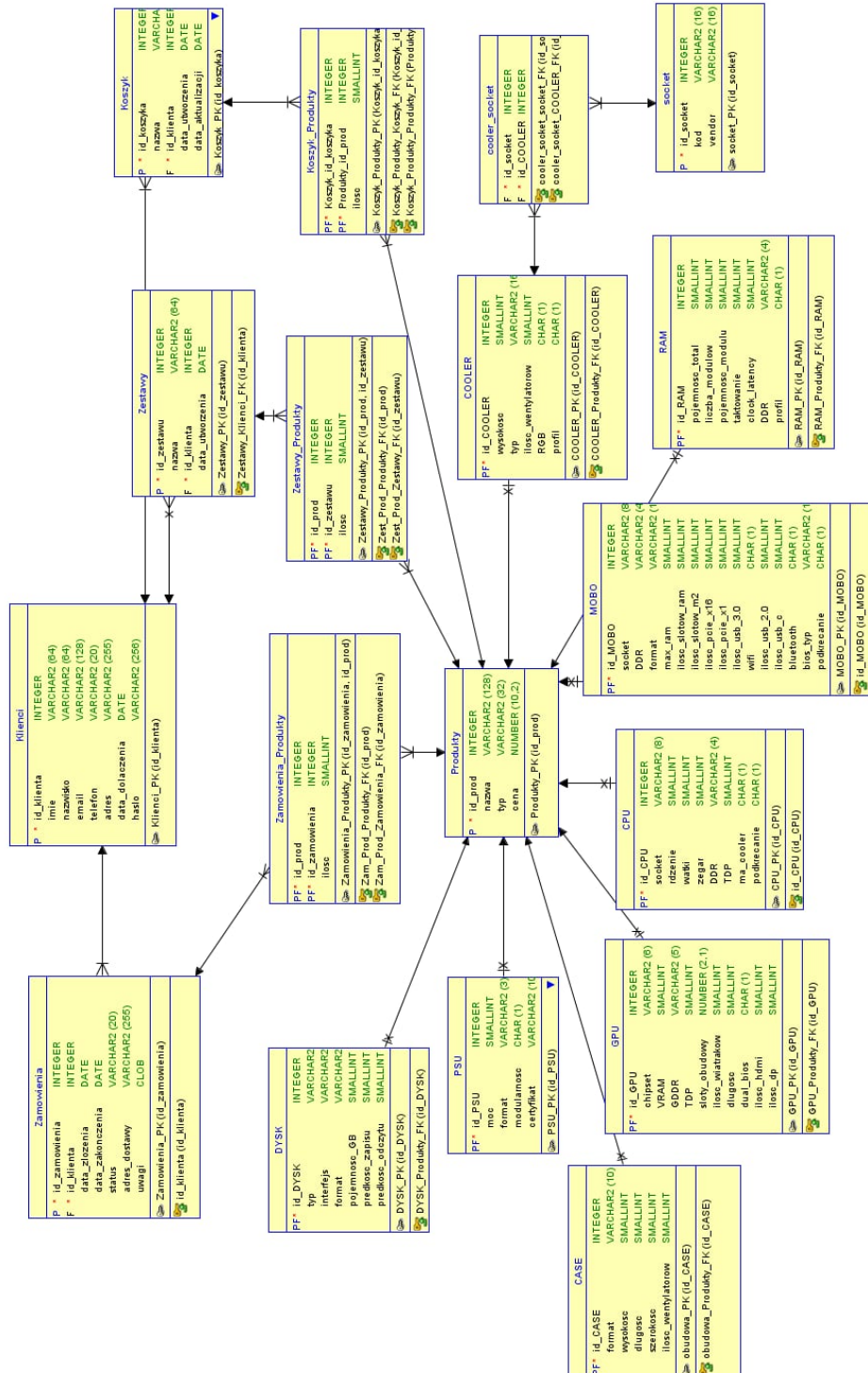
- ograniczenie ruchu sieciowego wyłącznie do testowej podsieci wirtualnej.

Dzięki powyższym zabiegom uruchomienie wszystkich niezbędnych usług udało się zrealizować w ramach przyznanego limitu 100 USD. Po wyczerpaniu kredytu Azure dalsze utrzymanie pełnej infrastruktury okazało się jednak niemożliwe - warstwa frontendowa aplikacji (Azure Static Web App) została wyłączona i strona nie jest już dostępna online. Baza danych (Azure Database for PostgreSQL) pozostała natomiast aktywna i wciąż można uzyskać do niej zdalny dostęp.

5.3 Baza danych

Sekcja ta przedstawia kompletny opis struktury bazy danych zastosowanej w projekcie. Model danych został opracowany w postaci diagramu encji-związków (ERD), który obrazuje wszystkie kluczowe encje systemu, relacje pomiędzy nimi oraz podstawową logikę przetwarzania i przechowywania danych. Diagram ERD, przedstawiony na rysunku 5.2, stanowi formalną reprezentację projektu logicznego bazy danych. Uwzględnia on zarówno encje odpowiadające podstawowym obiektom domenowym systemu, jak i encje pomocnicze, służące do realizacji relacji typu M:N, normalizacji danych oraz zapewnienia spójności referencyjnej. Zaprojektowana struktura bazy danych umożliwia jednoznaczne odwzorowanie procesów zachodzących w systemie, w szczególności w zakresie konfiguracji zestawów komputerowych, weryfikacji kompatybilności podzespołów oraz obsługi danych użytkowników i zamówień. Zastosowane klucze główne i obce pozwalają na zachowanie integralności danych oraz wspierają efektywne wykonywanie zapytań w warstwie aplikacyjnej.

5.3.1 Model danych - ERD



Rysunek 5.2: Diagram ERD bazy danych

Na rysunku przedstawiono diagram ERD bazy danych systemu sklepu komputerowego z kreatorem zestawów PC. Model odzwierciedla główne obszary funkcjonalne aplikacji

takie jak:

- zarządzanie klientami,
- zarządzanie produktami,
- zarządzanie koszykami,
- zarządzanie zamówieniami.

Centralne miejsce w diagramie zajmuje tabela Produkty, pełniąca rolę nadrzędnego rejestru wszystkich pozycji dostępnych w systemie. Zawiera ona podstawowe informacje o każdym produkcie, takie jak nazwa, typ oraz cena, natomiast szczegółowe parametry techniczne przechowywane są w dedykowanych tabelach odpowiadających konkretnym kategoriom sprzętu: CPU, GPU, RAM, MOBO, PSU, COOLER, DYSK oraz CASE. Każda z tych tabel posiada klucz obcy wskazujący na rekord w tabeli Produkty, co pozwala uniknąć redundancji danych oraz ułatwia zarządzanie wspólnymi atrybutami produktów. Relacje związane z procesem zakupowym odwzorowane są poprzez tabele Klienci, Koszyk, Zamówienia oraz Zestawy. Każdy klient może posiadać wiele koszyków, zamówień oraz zapisanych zestawów, co odzwierciedlono relacjami 1:N. Zawartość koszyka, zamówienia oraz zestawu została zrealizowana za pomocą tabel łączących Koszyk_Produkty, Zamówienia_Produkty oraz Zestawy_Produkty, które przechowują pary identyfikatorów produktu i obiektu nadrzędnego wraz z informacją o liczbie sztuk. Umożliwia to poprawne odwzorowanie relacji wiele-do-wielu pomiędzy produktami a koszykami, zamówieniami i zestawami.

Istotnym elementem modelu są tabele socket oraz cooler_socket, pełniące rolę słowników technicznych wykorzystywanych w logice kompatybilności. Tabela socket przechowuje informacje o typach gniazd procesorów, natomiast tabela cooler_socket łączy coolery z obsługiwanyimi przez nie socketami. Rozwiązanie to umożliwia powiązanie procesorów, płyt głównych oraz systemów chłodzenia oraz ułatwia implementację reguł sprawdzających zgodność komponentów.

Zaprojektowany model danych zachowuje spójność referencyjną dzięki zastosowaniu kluczy głównych i obcych, a rozdzielenie danych ogólnych i szczegółowych na osobne tabele umożliwia utrzymanie bazy w co najmniej trzeciej postaci normalnej. Takie podejście ułatwia dalszą rozbudowę systemu o nowe typy podzespołów oraz dodatkowe funkcje biznesowe, jednocześnie zapewniając integralność i wysoką jakość przechowywanych danych.

Opis przeznaczenia encji:

1. Tabela Klienci przechowuje dane identyfikacyjne użytkowników systemu. Zawiera informacje takie jak imię, nazwisko, adres e-mail, numer telefonu, adres zamieszkania oraz datę dołączenia do systemu. Tabela ta stanowi podstawę identyfikacji użytkowników i jest powiązana z tabelami Koszyki, Zamówienia oraz Zestawy.
2. Tabela Koszyki odpowiada za przechowywanie aktualnych koszyków zakupowych użytkowników. Każdy rekord reprezentuje pojedynczy koszyk przypisany do konkretnego klienta. Tabela zawiera informacje o dacie utworzenia oraz ostatniej aktualizacji koszyka i umożliwia przechowywanie tymczasowych konfiguracji produktów przed złożeniem zamówienia.
3. Tabela Koszyk_Produkty jest tabelą pośredniczącą realizującą relację wiele-do-wielu pomiędzy tabelami Koszyki oraz Produkty. Przechowuje identyfikatory koszyka i produktu wraz z informacją o aktualnie przechowywanej liczbie sztuk danego produktu. Rozwiązanie to umożliwia elastyczne i dynamiczne zarządzanie zawartością koszyka.
4. Tabela Zamówienia przechowuje informacje o zamówieniach złożonych przez użytkowników. Zawiera dane takie jak data złożenia zamówienia, data zakończenia realizacji, status zamówienia, adres dostawy oraz dodatkowe uwagi. Każde zamówienie jest przypisane do konkretnego klienta.
5. Tabela Zamówienia_Produkty realizuje relację wiele-do-wielu pomiędzy tabelami Zamówienia oraz Produkty. Przechowuje informacje o produktach wchodzących w skład zamówienia oraz ich liczbach, umożliwiając poprawne odwzorowanie struktury zamówienia.
6. Tabela Zestawy przechowuje konfiguracje zestawów komputerowych tworzone przez użytkowników. Każdy zestaw posiada nazwę, datę utworzenia oraz jest przypisany do konkretnego klienta. Tabela ta umożliwia zapisywanie i ponowne wykorzystanie konfiguracji zestawów.
7. Tabela Zestawy_Produkty jest tabelą pośredniczącą odwzorowującą relację wiele-do-wielu pomiędzy tabelami Zestawy oraz Produkty. Przechowuje identyfikatory zestawu i produktu wraz z informacją o liczbie sztuk, co pozwala na budowę pełnej konfiguracji zestawu komputerowego.

5.3.2 Opis najważniejszych encji

Tabela produkty

Najbardziej istotną, pełniącą funkcję głównej, tabelą w całej bazie danych jest tabela produkty. Kluczem tabeli jest `id_prod`, klucz ten podstawą działania aplikacji. Encja produkty zawiera również pełną nazwę produktu w typie `varchar`, typ produktu, który umożliwia prowadzenie poprawnych operacji w kreatorze zestawów, stanowiąc pole odpowiedzialne za łączenie odpowiednich tabel z dodatkowymi danymi konkretnego produktu, zależnie od jego rodzaju. Tabela zawiera również kolumnę `cena` w typie `numeric`, który wybrano w celu zachowania spójności działania programu. Strukturę tabeli Zestawy przedstawiono w tabeli 5.1.

Tabela 5.1: Struktura tabeli Produkty

Nazwa pola	Typ danych	Opis
<code>id_prod</code>	<code>integer</code>	Klucz główny tabeli
<code>nazwa</code>	<code>varchar(128)</code>	Nazwa produktu
<code>typ</code>	<code>varchar(32)</code>	Typ podzespołu
<code>cena</code>	<code>numeric(10,2)</code>	Cena produktu

Poniżej przedstawiono opis poszczególnych pól wchodzących w skład tabeli Produkty:

1. Pole `id_prod` jest kluczem głównym tabeli Produkty. Służy do jednoznacznej identyfikacji każdego produktu w systemie oraz stanowi podstawę powiązań z pozostałymi tabelami bazy danych.
2. Atrybut `nazwa` przechowuje pełną nazwę produktu w postaci tekstowej. Wartość ta jest wykorzystywana w warstwie frontendowej do prezentacji informacji użytkownikowi oraz w backendzie podczas obsługi logiki biznesowej.
3. Pole `typ` określa kategorię produktu, taką jak procesor, karta graficzna czy pamięć RAM. Atrybut ten umożliwia rozróżnienie rodzaju podzespołu oraz poprawne powiązanie rekordu z odpowiednią tabelą zawierającą szczegółowe parametry techniczne.
4. Atrybut `cena` przechowuje aktualną cenę produktu w postaci liczby typu numerycznego. Zastosowanie tego typu danych umożliwia precyzyjne operacje obliczeniowe związane z wyliczaniem wartości koszyka oraz zamówień.

Tabela Klienci

Tabela Klienci przechowuje dane identyfikacyjne użytkowników systemu oraz informacje niezbędne do realizacji procesów zakupowych. Encja ta stanowi podstawę obsługi użytkowników aplikacji i jest powiązana z tabelami Koszyki, Zamówienia oraz Zestawy. Dane zgromadzone w tabeli umożliwiają jednoznaczną identyfikację klienta oraz przypisanie mu odpowiednich zasobów systemowych. Strukturę tabeli Zestawy przedstawiono w tabeli 5.2.

Tabela 5.2: Struktura tabeli Klienci

Nazwa pola	Typ danych	Opis
id_klienta	integer	Klucz główny tabeli
imie	varchar(64)	Imię klienta
nazwisko	varchar(64)	Nazwisko klienta
email	varchar(128)	Adres poczty elektronicznej
telefon	varchar(20)	Numer telefonu kontaktowego
adres	varchar(255)	Adres zamieszkania klienta
data_dolaczenia	date	Data rejestracji w systemie
haslo	varchar(256)	Zaszyfrowane hasło użytkownika

Poniżej przedstawiono opis poszczególnych pól wchodzących w skład tabeli Klienci:

1. Pole id_klienta jest kluczem głównym tabeli Klienci. Służy do jednoznacznej identyfikacji użytkownika w systemie oraz stanowi podstawę powiązań z innymi tabelami bazy danych.
2. Atrybut imie przechowuje imię klienta w postaci tekstowej. Informacja ta wykorzystywana jest głównie do celów prezentacyjnych w interfejsie użytkownika.
3. Pole nazwisko zawiera nazwisko klienta. Wraz z polem imie umożliwia pełną identyfikację użytkownika w systemie.
4. Atrybut email przechowuje adres poczty elektronicznej klienta, który wykorzystywany jest jako dane kontaktowe oraz może pełnić rolę identyfikatora podczas procesu logowania.
5. Pole telefon zawiera numer telefonu kontaktowego użytkownika. Pole to umożliwia dodatkową formę kontaktu z klientem w ramach realizacji zamówień.
6. Atrybut adres przechowuje adres zamieszkania klienta, wykorzystywany w procesie realizacji dostawy zamówionych produktów.
7. Pole data_dolaczenia określa datę rejestracji użytkownika w systemie. Domyślna wartość pola ustawiana jest na aktualną datę w momencie utworzenia rekordu.

8. Atrybut hasło przechowuje hasło użytkownika w postaci zaszyfrowanej. Pole to wykorzystywane jest w procesie uwierzytelniania i zapewnia bezpieczeństwo dostępu do konta użytkownika.

Tabela Koszyk

Tabela Koszyk przechowuje informacje o koszykach zakupowych użytkowników systemu. Każdy koszyk przypisany jest do konkretnego klienta i służy do tymczasowego gromadzenia wybranych produktów przed złożeniem zamówienia. Encja ta umożliwia zarządzanie zawartością koszyka oraz śledzenie zmian dokonywanych przez użytkownika. Strukturę tabeli Zestawy przedstawiono w tabeli 5.3.

Tabela 5.3: Struktura tabeli Koszyk

Nazwa pola	Typ danych	Opis
id_koszyka	integer	Klucz główny tabeli
nazwa	varchar(64)	Nazwa koszyka
id_klienta	integer	Identyfikator klienta
data_utworzenia	date	Data utworzenia koszyka
data_aktualizacji	date	Data ostatniej aktualizacji koszyka

Poniżej przedstawiono opis poszczególnych pól wchodzących w skład tabeli Koszyk:

1. Pole id_koszyka jest kluczem głównym tabeli Koszyk. Służy do jednoznacznej identyfikacji koszyka w systemie oraz stanowi podstawę powiązań z tabelą Koszyk_Produkty.
2. Atrybut nazwa przechowuje nazwę koszyka, która może być wykorzystywana do rozróżniania wielu koszyków przypisanych do jednego użytkownika.
3. Pole id_klienta jest kluczem obcym wskazującym na tabelę Klienci. Pole to umożliwia jednoznaczne przypisanie koszyka do konkretnego użytkownika systemu.
4. Atrybut data_utworzenia określa datę utworzenia koszyka w systemie. Informacja ta może być wykorzystywana do celów analitycznych oraz porządkowych.
5. Pole data_aktualizacji przechowuje datę ostatniej modyfikacji koszyka. Pole to pozwala na śledzenie aktywności użytkownika oraz zmian wprowadzanych w zawartości koszyka.

Tabela Zamówienia

Tabela Zamówienia przechowuje informacje dotyczące zamówień składanych przez użytkowników systemu. Każdy rekord reprezentuje pojedyncze zamówienie przypisane do konkretnego klienta i stanowi finalny etap procesu zakupowego. Encja ta umożliwia śledzenie przebiegu realizacji zamówienia oraz przechowywanie danych niezbędnych do jego obsługi. Strukturę tabeli Zestawy przedstawiono w tabeli 5.4.

Tabela 5.4: Struktura tabeli Zamówienia

Nazwa pola	Typ danych	Opis
id_zamowienia	integer	Klucz główny tabeli
id_klienta	integer	Identyfikator klienta
data_zlozenia	date	Data złożenia zamówienia
data_zakonczenia	date	Data zakończenia realizacji zamówienia
status	varchar(20)	Status zamówienia
adres_dostawy	varchar(255)	Adres dostawy
uwagi	text	Dodatkowe uwagi do zamówienia

Poniżej przedstawiono opis poszczególnych pól tabeli Zamówienia:

1. Pole `id_zamowienia` jest kluczem głównym tabeli Zamówienia. Służy do jednoznacznej identyfikacji zamówienia w systemie oraz stanowi podstawę powiązań z tabelą `Zamówienia_Produkty`.
2. Atrybut `id_klienta` jest kluczem obcym wskazującym na tabelę `Klienci`. Umożliwia przypisanie zamówienia do konkretnego użytkownika systemu.
3. Pole `data_zlozenia` określa datę złożenia zamówienia przez klienta. Informacja ta wykorzystywana jest do monitorowania czasu realizacji zamówienia.
4. Atrybut `data_zakonczenia` przechowuje datę zakończenia realizacji zamówienia. Pole to pozwala na określenie momentu finalizacji procesu zakupowego.
5. Pole `status` określa aktualny stan zamówienia, na przykład złożone, w trakcie realizacji lub zakończone. Umożliwia to kontrolę przebiegu realizacji zamówień.
6. Atrybut `adres_dostawy` przechowuje adres, na który mają zostać dostarczone zamówione produkty. Informacja ta jest niezbędna do poprawnej realizacji procesu logistycznego.
7. Pole `uwagi` umożliwia zapisanie dodatkowych informacji lub komentarzy dotyczących zamówienia, przekazanych przez użytkownika.

Tabela Zestawy

Tabela Zestawy przechowuje informacje o zestawach komputerowych tworzonych przez użytkowników systemu. Każdy rekord reprezentuje pojedynczy zestaw przypisany do konkretnego klienta i umożliwia zapisywanie oraz ponowne wykorzystanie konfiguracji podzespołów. Encja ta stanowi element kreatora zestawów komputerowych. Strukturę tabeli Zestawy przedstawiono w tabeli 5.5.

Tabela 5.5: Struktura tabeli Zestawy

Nazwa pola	Typ danych	Opis
id_zestawu	integer	Klucz główny tabeli
nazwa	varchar(64)	Nazwa zestawu
id_klienta	integer	Identyfikator klienta
data_utworzenia	date	Data utworzenia zestawu

Poniżej przedstawiono opis poszczególnych pól tabeli Zestawy:

1. Atrybut `id_zestawu` jest kluczem głównym tabeli Zestawy. Służy do jednoznacznej identyfikacji zestawu w systemie oraz stanowi podstawę powiązań z tabelą `Zestawy_Produkty`.
2. Atrybut `nazwa` przechowuje nazwę zestawu nadaną przez użytkownika. Umożliwia ona rozróżnienie wielu konfiguracji zapisanych przez tego samego klienta.
3. Atrybut `id_klienta` jest kluczem obcym wskazującym na tabelę `Klienci`. Pozwala na przypisanie zestawu do konkretnego użytkownika systemu.
4. Atrybut `data_utworzenia` określa datę utworzenia zestawu komputerowego. Informacja ta może być wykorzystywana do celów porządkowych oraz analitycznych.

Tabela GPU

Tabela GPU przechowuje szczegółowe informacje techniczne dotyczące kart graficznych dostępnych w systemie. Encja ta zawiera parametry istotne z punktu widzenia kompatybilności zestawu komputerowego, takie jak zapotrzebowanie energetyczne, wymiary fizyczne oraz liczba obsługiwanych interfejsów wyjściowych. Tabela GPU jest powiązana z tabelą Produkty i uzupełnia ją o dane specyficzne dla kart graficznych. Strukturę tabeli GPU przedstawiono w tabeli 5.6.

Tabela 5.6: Struktura tabeli GPU

Nazwa pola	Typ danych	Opis
id_gpu	integer	Klucz główny tabeli
chipset	varchar(6)	Rodzina lub seria układu graficznego
vram	smallint	Pojemność pamięci VRAM w gigabajtach
gddr	varchar(6)	Typ zastosowanej pamięci graficznej
tdp	smallint	Zapotrzebowanie energetyczne karty
sloty_obudowy	numeric(2,1)	Liczba slotów zajmowanych w obudowie
ilosc_wiatrakow	smallint	Liczba wentylatorów karty graficznej
dlugosc	smallint	Długość karty graficznej
dual_bios	smallint	Informacja o obsłudze trybu dual BIOS
ilosc_hdmi	smallint	Liczba wyjść HDMI
ilosc_dp	smallint	Liczba wyjść DisplayPort

Poniżej przedstawiono opis poszczególnych pól tabeli GPU:

1. Atrybut id_gpu jest kluczem głównym tabeli GPU. Służy do jednoznacznej identyfikacji karty graficznej oraz stanowi podstawę powiązań z tabelą Produkty.
2. Atrybut chipset określa rodzinę lub serię układu graficznego. Informacja ta wykorzystywana jest podczas filtrowania oraz prezentacji produktów.
3. Atrybut vram przechowuje pojemność pamięci graficznej karty. Parametr ten ma istotne znaczenie przy ocenie zestawu.
4. Atrybut gddr określa typ zastosowanej pamięci graficznej, na przykład GDDR6 lub GDDR6X.
5. Atrybut tdp przechowuje informację o zapotrzebowaniu energetycznym karty graficznej. Dane te wykorzystywane są podczas analizy kompatybilności z zasilaczem.
6. Atrybut sloty_obudowy określa liczbę slotów zajmowanych przez kartę w obudowie komputera. Parametr ten jest istotny przy weryfikacji zgodności z obudową.
7. Atrybut ilosc_wiatrakow przechowuje liczbę wentylatorów zastosowanych w układzie chłodzenia karty graficznej.

8. Atrybut `dlugosc` określa długość fizyczną karty graficznej. Wartość ta wykorzystywana jest w procesie sprawdzania kompatybilności z obudową.
9. Atrybut `dual_bios` informuje o obecności trybu podwójnego BIOS-u. Umożliwia to rozróżnienie kart oferujących dodatkowe funkcje konfiguracyjne.
10. Atrybut `ilosc_hdmi` określa liczbę dostępnych wyjść HDMI.
11. Atrybut `ilosc_dp` określa liczbę dostępnych wyjść DisplayPort.

Tabela CPU

Tabela CPU przechowuje szczegółowe informacje techniczne dotyczące procesorów dostępnych w systemie. Encja ta zawiera parametry istotne z punktu widzenia wydajności oraz kompatybilności zestawu komputerowego, takie jak typ gniazda, liczba rdzeni i wątków, zapotrzebowanie energetyczne oraz obsługa dodatkowych funkcji. Tabela CPU jest powiązana z tabelą Produkty i uzupełnia ją o dane specyficzne dla procesorów. Strukturę tabeli CPU przedstawiono w tabeli 5.7.

Tabela 5.7: Struktura tabeli CPU

Nazwa pola	Typ danych	Opis
id_cpu	integer	Klucz główny tabeli
socket	varchar(8)	Typ gniazda procesora
rdzenie	smallint	Liczba rdzeni procesora
watki	smallint	Liczba wątków procesora
zegar	smallint	Częstotliwość taktowania bazowego
tdp	smallint	Zapotrzebowanie energetyczne procesora
ma_cooler	smallint	Informacja o dołączonym chłodzeniu
podkrecanie	smallint	Obsługa podkręcania
ma_integre	smallint	Informacja o zintegrowanym układzie graficznym

Poniżej przedstawiono opis poszczególnych pól tabeli CPU:

1. Atrybut id_cpu jest kluczem głównym tabeli CPU. Służy do jednoznacznej identyfikacji procesora oraz stanowi podstawę powiązań z tabelą Produkty.
2. Atrybut socket określa typ gniazda procesora. Parametr ten jest wykorzystywany podczas sprawdzania kompatybilności procesora z płytą główną oraz systemem chłodzenia.
3. Atrybut rdzenie przechowuje liczbę fizycznych rdzeni procesora, co ma bezpośredni wpływ na jego wydajność.
4. Atrybut watki określa liczbę obsługiwanych wątków. Informacja ta wykorzystywana jest do oceny możliwości procesora w zastosowaniach wielowątkowych.
5. Atrybut zegar przechowuje częstotliwość taktowania bazowego procesora. Parametr ten ma znaczenie przy porównywaniu wydajności jednostek obliczeniowych.
6. Atrybut tdp określa zapotrzebowanie energetyczne procesora. Dane te wykorzystywane są w procesie doboru odpowiedniego zasilacza oraz systemu chłodzenia.
7. Atrybut ma_cooler informuje, czy do procesora dołączone jest fabryczne chłodzenie.
8. Atrybut podkrecanie określa możliwość podkręcania procesora. Informacja ta pozwala na rozróżnienie modeli oferujących dodatkowe możliwości konfiguracyjne.

9. Atrybut `ma_integre` informuje o obecności zintegrowanego układu graficznego w procesorze.

Tabela MOBO

Tabela MOBO przechowuje szczegółowe informacje techniczne dotyczące płyt głównych dostępnych w systemie. Encja ta stanowi kluczowy element procesu weryfikacji kompatybilności, ponieważ łączy parametry procesora, pamięci RAM oraz pozostałych podzespołów zestawu komputerowego. Tabela MOBO jest powiązana z tabelą Produkty i uzupełnia ją o dane specyficzne dla płyt głównych. Strukturę tabeli MOBO przedstawiono w tabeli 5.8.

Tabela 5.8: Struktura tabeli MOBO

Nazwa pola	Typ danych	Opis
<code>id_mobo</code>	<code>integer</code>	Klucz główny tabeli
<code>socket</code>	<code>varchar(8)</code>	Typ gniazda procesora
<code>ddr</code>	<code>varchar(4)</code>	Obsługiwany standard pamięci RAM
<code>format</code>	<code>varchar(10)</code>	Format płyty głównej
<code>max_ram</code>	<code>smallint</code>	Maksymalna obsługiwana ilość pamięci RAM
<code>ilosc_slotow_ram</code>	<code>smallint</code>	Liczba slotów pamięci RAM
<code>ilosc_slotow_m2</code>	<code>smallint</code>	Liczba slotów M.2
<code>ilosc_pcie_x16</code>	<code>smallint</code>	Liczba złączy PCIe x16
<code>ilosc_pcie_x1</code>	<code>smallint</code>	Liczba złączy PCIe x1
<code>ilosc_usb_3_0</code>	<code>smallint</code>	Liczba portów USB 3.0
<code>wifi</code>	<code>smallint</code>	Informacja o obsłudze Wi-Fi
<code>ilosc_usb_2_0</code>	<code>smallint</code>	Liczba portów USB 2.0
<code>ilosc_usb_c</code>	<code>smallint</code>	Liczba portów USB typu C
<code>bluetooth</code>	<code>smallint</code>	Informacja o obsłudze Bluetooth
<code>bios_typ</code>	<code>varchar(10)</code>	Typ BIOS-u
<code>podkrecanie</code>	<code>smallint</code>	Obsługa podkreśniania

Poniżej przedstawiono opis poszczególnych pól tabeli MOBO:

1. Atrybut `id_mobo` jest kluczem głównym tabeli MOBO. Służy do jednoznacznej identyfikacji płyty głównej oraz stanowi podstawę powiązań z tabelą Produkty.
2. Atrybut `socket` określa typ gniazda procesora. Parametr ten wykorzystywany jest podczas sprawdzania kompatybilności płyty głównej z procesorem.
3. Atrybut `ddr` przechowuje informację o obsługiwanym standardzie pamięci RAM, co umożliwia weryfikację zgodności z wybranymi modułami pamięci.
4. Atrybut `format` określa format płyty głównej, na przykład ATX lub mATX. Informacja ta jest wykorzystywana przy sprawdzaniu kompatybilności z obudową.

5. Atrybut `max_ram` określa maksymalną ilość pamięci RAM obsługiwaną przez płytę główną.
6. Atrybut `ilosc_slotow_ram` przechowuje liczbę slotów pamięci RAM dostępnych na płycie głównej.
7. Atrybut `ilosc_slotow_m2` określa liczbę dostępnych złączy M.2.
8. Atrybut `ilosc_pcie_x16` przechowuje liczbę złączy PCIe x16, wykorzystywanych głównie do podłączania kart graficznych.
9. Atrybut `ilosc_pcie_x1` określa liczbę złączy PCIe x1 przeznaczonych dla dodatkowych kart rozszerzeń.
10. Atrybut `ilosc_usb_3_0` przechowuje liczbę portów USB 3.0 dostępnych na płycie głównej.
11. Atrybut `wifi` informuje o obecności modułu Wi-Fi na płycie głównej.
12. Atrybut `ilosc_usb_2_0` określa liczbę portów USB 2.0.
13. Atrybut `ilosc_usb_c` przechowuje liczbę portów USB typu C.
14. Atrybut `bluetooth` informuje o obsłudze technologii Bluetooth.
15. Atrybut `bios_typ` określa typ zastosowanego BIOS-u.
16. Atrybut `podkrecanie` informuje o możliwości podkręcania podzespołów przy użyciu danej płyty głównej.

Tabela RAM

Tabela RAM przechowuje szczegółowe informacje techniczne dotyczące pamięci operacyjnej wykorzystywanej w zestawach komputerowych. Encja ta zawiera parametry istotne z punktu widzenia kompatybilności oraz wydajności systemu, takie jak pojemność, taktowanie oraz standard pamięci. Tabela RAM jest powiązana z tabelą Produkty i uzupełnia ją o dane specyficzne dla modułów pamięci operacyjnej. Strukturę tabeli RAM przedstawiono w tabeli 5.9.

Tabela 5.9: Struktura tabeli RAM

Nazwa pola	Typ danych	Opis
id_ram	integer	Klucz główny tabeli
pojemnosc_total	smallint	Łączna pojemność pamięci RAM
liczba_modulow	smallint	Liczba modułów pamięci
pojemnosc_modulu	smallint	Pojemność pojedynczego modułu
taktowanie	smallint	Częstotliwość taktowania pamięci
clock_latency	smallint	Opóźnienie pamięci (CL)
ddr	varchar(4)	Standard pamięci RAM
profil	char(1)	Informacja o profilu XMP/EXPO

Poniżej przedstawiono opis poszczególnych pól tabeli RAM:

1. Atrybut `id_ram` jest kluczem głównym tabeli RAM. Służy do jednoznacznej identyfikacji modułu pamięci operacyjnej oraz stanowi podstawę powiązań z tabelą `Produkty`.
2. Atrybut `pojemnosc_total` określa całkowitą pojemność pamięci RAM dostępnej w zestawie.
3. Atrybut `liczba_modulow` przechowuje informację o liczbie modułów pamięci wchodzących w skład zestawu RAM.
4. Atrybut `pojemnosc_modulu` określa pojemność pojedynczego modułu pamięci operacyjnej.
5. Atrybut `taktowanie` przechowuje częstotliwość taktowania pamięci RAM. Parametr ten ma istotne znaczenie dla wydajności systemu.
6. Atrybut `clock_latency` określa opóźnienie pamięci RAM, oznaczane jako CL. Informacja ta wykorzystywana jest przy porównywaniu parametrów modułów.
7. Atrybut `ddr` określa standard pamięci RAM, na przykład DDR4 lub DDR5. Parametr ten wykorzystywany jest podczas sprawdzania kompatybilności z płytą główną.
8. Atrybut `profil` informuje o obsłudze profilu pamięci, takiego jak XMP lub EXPO, umożliwiającego pracę modułów z podwyższonymi parametrami.

Tabela PSU

Tabela PSU przechowuje informacje techniczne dotyczące zasilaczy komputerowych dostępnych w systemie. Encja ta zawiera parametry istotne z punktu widzenia stabilności oraz bezpieczeństwa zestawu komputerowego, w szczególności moc zasilacza, jego format oraz certyfikację energetyczną. Tabela PSU jest powiązana z tabelą Produkty i uzupełnia ją o dane specyficzne dla zasilaczy. Strukturę tabeli PSU przedstawiono w tabeli 5.10.

Tabela 5.10: Struktura tabeli PSU

Nazwa pola	Typ danych	Opis
id_psu	integer	Klucz główny tabeli
moc	smallint	Moc znamionowa zasilacza
format	varchar(3)	Format zasilacza
modularnosc	smallint	Informacja o modularności okablowania
certyfikat	varchar(10)	Certyfikat sprawności energetycznej

Poniżej przedstawiono opis poszczególnych pól tabeli PSU:

1. Atrybut `id_psu` jest kluczem głównym tabeli PSU. Służy do jednoznacznej identyfikacji zasilacza oraz stanowi podstawę powiązań z tabelą Produkty.
2. Atrybut `moc` określa moc znamionową zasilacza wyrażoną w watach. Parametr ten wykorzystywany jest podczas analizy zapotrzebowania energetycznego zestawu komputerowego.
3. Atrybut `format` określa standard obudowy zasilacza, na przykład ATX. Informacja ta ma znaczenie przy sprawdzaniu kompatybilności z obudową komputera.
4. Atrybut `modularnosc` informuje, czy zasilacz posiada modularne okablowanie. Parametr ten wpływa na estetykę oraz łatwość zarządzania przewodami w obudowie.
5. Atrybut `certyfikat` przechowuje informację o klasie sprawności energetycznej zasilacza, na przykład 80 PLUS Bronze, Gold lub Platinum.

Tabela Cooler

Tabela COOLER przechowuje informacje techniczne dotyczące systemów chłodzenia procesora dostępnych w systemie. Encja ta zawiera parametry istotne z punktu widzenia kompatybilności mechanicznej oraz efektywności chłodzenia, takie jak wysokość chłodzenia, jego typ oraz liczba wentylatorów. Tabela COOLER jest powiązana z tabelą Produkty i uzupełnia ją o dane specyficzne dla systemów chłodzenia. Strukturę tabeli COOLER przedstawiono w tabeli 5.11.

Tabela 5.11: Struktura tabeli COOLER

Nazwa pola	Typ danych	Opis
id_cooler	integer	Klucz główny tabeli
wysokosc	smallint	Wysokość systemu chłodzenia
typ	varchar(16)	Typ chłodzenia
ilosc_wentylatorow	smallint	Liczba wentylatorów
rgb	smallint	Informacja o podświetleniu RGB
profil	smallint	Profil pracy chłodzenia

Poniżej przedstawiono opis poszczególnych pól tabeli COOLER:

1. Atrybut id_cooler jest kluczem głównym tabeli COOLER. Służy do jednoznacznej identyfikacji systemu chłodzenia oraz stanowi podstawę powiązań z tabelą Produkty.
2. Atrybut wysokosc określa wysokość systemu chłodzenia. Parametr ten wykorzystywany jest podczas sprawdzania kompatybilności chłodzenia z obudową komputera.
3. Atrybut typ przechowuje informację o rodzaju systemu chłodzenia, na przykład powietrzne lub ciecżą.
4. Atrybut ilosc_wentylatorow określa liczbę wentylatorów zastosowanych w systemie chłodzenia, co wpływa na jego wydajność.
5. Atrybut rgb informuje o obecności podświetlenia RGB w systemie chłodzenia.
6. Atrybut profil określa profil pracy systemu chłodzenia, umożliwiając dostosowanie jego działania do preferencji użytkownika.

Tabela Dysk

Tabela DYSK przechowuje informacje techniczne dotyczące nośników danych wykorzystywanych w zestawach komputerowych. Encja ta zawiera parametry istotne z punktu widzenia wydajności oraz kompatybilności systemu, takie jak typ nośnika, zastosowany interfejs, format fizyczny oraz prędkości odczytu i zapisu. Tabela DYSK jest powiązana z tabelą Produkty i uzupełnia ją o dane specyficzne dla urządzeń pamięci masowej. Strukturę tabeli DYSK przedstawiono w tabeli 5.12.

Tabela 5.12: Struktura tabeli DYSK

Nazwa pola	Typ danych	Opis
id_dysk	integer	Klucz główny tabeli
typ	varchar(5)	Typ nośnika danych
interfejs	varchar(12)	Zastosowany interfejs
format	varchar(12)	Format fizyczny nośnika
pojemnosc_gb	smallint	Pojemność nośnika danych
predkosc_zapisu	smallint	Prędkość zapisu danych
predkosc_odczytu	smallint	Prędkość odczytu danych

Poniżej przedstawiono opis poszczególnych pól tabeli DYSK:

1. Atrybut id_dysk jest kluczem głównym tabeli DYSK. Służy do jednoznacznej identyfikacji nośnika danych oraz stanowi podstawę powiązań z tabelą Produkty.
2. Atrybut typ określa rodzaj nośnika danych, na przykład HDD lub SSD.
3. Atrybut interfejs przechowuje informację o zastosowanym interfejsie, takim jak SATA lub NVMe, co umożliwia sprawdzenie kompatybilności z płytą główną.
4. Atrybut format określa format fizyczny nośnika danych, na przykład 2.5 cala lub M.2.
5. Atrybut pojemnosc_gb przechowuje pojemność nośnika danych wyrażoną w gigabajtach.
6. Atrybut predkosc_zapisu określa maksymalną prędkość zapisu danych.
7. Atrybut predkosc_odczytu określa maksymalną prędkość odczytu danych.

5.4 Komponenty i moduły systemu

System został zaprojektowany w sposób modułowy, co umożliwia jego czytelną strukturę oraz łatwą rozbudowę w przyszłości. Poszczególne moduły realizują konkretne zadania funkcjonalne systemu, a ich współdziałanie odbywa się po stronie backendu aplikacji.

5.4.1 Moduł kompatybilności podzespołów

Moduł kompatybilności podzespołów odpowiada za weryfikację technicznej zgodności komponentów wybieranych przez użytkownika podczas budowy zestawu komputerowego. Proces weryfikacji realizowany jest etapowo, zgodnie z określoną kolejnością reguł.

```
def check_cpu_mobo(cpu, motherboard):
    return cpu.socket == motherboard.socket
```

Pierwszym etapem weryfikacji jest sprawdzenie zgodności gniazda procesora z gniazdem płyty głównej. Jest to kluczowa reguła kompatybilności, bez której dalsza konfiguracja zestawu nie jest możliwa.

```
def check_ram_ddr(ram, motherboard):
    return ram.-ddr_type == motherboard.-ddr_type
```

Kolejnym krokiem jest sprawdzenie zgodności standardu pamięci RAM z płytą główną. Moduł analizuje typ pamięci DDR oraz obsługiwany standard przez płytę główną.

```
def check_power(cpu, gpu, psu):
    total_power = cpu.tdp + gpu.tdp
    return psu.power >= total_power
```

Dodatkowo weryfikowane jest zapotrzebowanie energetyczne zestawu w odniesieniu do mocy zasilacza. W przypadku wykrycia niezgodności system blokuje możliwość zapisania zestawu.

5.4.2 Moduł logiki rozmytej

Moduł logiki rozmytej został zastosowany w celu elastycznej klasyfikacji zestawów komputerowych do określonych kategorii użytkowych. W przeciwieństwie do klasycznych algorytmów decyzyjnych opartych na ostrych progach, zaproponowane rozwiązanie umożliwia ocenę konfiguracji w sposób niebinarny, z uwzględnieniem stopnia dopasowania zestawu do poszczególnych kategorii.

Podstawą działania modułu są funkcje przynależności zbiorów rozmytych, opisujące takie cechy zestawu jak łączna cena, wydajność oraz zapotrzebowanie energetyczne. Do opisu przynależności zastosowano trapezoidalną funkcję przynależności, która umożliwia płynne przejście pomiędzy stanami częściowej i pełnej przynależności.

```
def trapezoidal_membership(x, a, x1, x2, b):
    if x <= a or x >= b:
        return 0.0
```

```
elif a < x < x1:
    return (x - a) / (x1 - a)
elif x1 <= x <= x2:
    return 1.0
elif x2 < x < b:
    return (b - x) / (b - x2)
else:
    return 0.0
```

Na podstawie obliczonych stopni przynależności realizowany jest proces wnioskowania rozmytego typu Mamdaniego, w którym operator minimum pełni rolę koniunkcji logicznej. Wyniki poszczególnych reguł są następnie agregowane, a ostateczna klasyfikacja zestawu wyznaczana jest na podstawie kategorii o najwyższym stopniu dopasowania.

```
def classify_build_fuzzy(price, performance, power):
    mu_budget = trapezoidal_membership(price, 0, 1500, 3000, 5000)
    mu_performance = trapezoidal_membership(performance, 50, 70, 90, 100)
    mu_energy = trapezoidal_membership(power, 0, 200, 400, 700)

    score_budget = min(mu_budget, 1 - mu_energy)
    score_performance = min(mu_performance, mu_energy)
    score_energy = min(1 - mu_performance, mu_energy)

    scores = {
        "budzetowy": score_budget,
        "wydajny": score_performance,
        "energooszczędny": score_energy
    }

    return max(scores, key=scores.get), scores
```

Zastosowane podejście pozwala na uwzględnienie przypadków pośrednich, w których zestaw komputerowy nie spełnia jednoznacznie kryteriów jednej kategorii. Dzięki temu system generuje bardziej realistyczne i użyteczne rekomendacje, zwiększając jakość wsparcia decyzyjnego dla użytkownika końcowego.

Zależność pomiędzy wartością analizowanej cechy a stopniem jej przynależności do zbioru rozmytego opisano następującą zależnością:

$$\mu(x) = \begin{cases} 0 & x \leq a \\ \frac{x-a}{x_1-a} & a < x < x_1 \\ 1 & x_1 \leq x \leq x_2 \\ \frac{b-x}{b-x_2} & x_2 < x < b \\ 0 & x \geq b \end{cases}$$

gdzie:

- x – wartość analizowanej cechy wejściowej, na przykład znormalizowana cena zestawu komputerowego,
- a – dolna granica przedziału, dla której stopień przynależności do zbioru rozmytego wynosi 0,
- x_1 – punkt, w którym funkcja przynależności osiąga wartość maksymalną 1,
- x_2 – punkt, do którego stopień przynależności utrzymuje wartość maksymalną 1,
- b – górna granica przedziału, po przekroczeniu której stopień przynależności ponownie przyjmuje wartość 0.

5.4.3 Moduł zarządzania koszykiem i zamówieniami

Moduł zarządzania koszykiem odpowiada za obsługę procesów związanych z gromadzeniem produktów wybranych przez użytkownika oraz przygotowaniem ich do realizacji zamówienia.

```
@router.post("/cart/add")
def add_to_cart(product_id: int, quantity: int = 1):
    item = get_cart_item(product_id)
    if item:
        item.quantity += quantity
    else:
        create_cart_item(product_id, quantity)
```

System umożliwia dodawanie produktów do koszyka oraz automatyczne zwiększanie ilości w przypadku ponownego dodania tego samego produktu.

```
@router.get("/cart")
def get_cart():
    return fetch_cart_items()
```

Zawartość koszyka może zostać pobrana w dowolnym momencie, co pozwala na bieżące wyświetlanie aktualnego stanu koszyka w interfejsie użytkownika.

```
@router.delete("/cart/clear")
def clear_cart():
    remove_all_items()
```

Moduł umożliwia również całkowite wyczyszczenie koszyka, na przykład po zakończeniu procesu zakupowego.

5.5 Najważniejsze algorytmy

Jednym z kluczowych algorytmów systemu jest algorytm sprawdzania kompatybilności podzespołów. Algorytm jest wykonywany sekwencyjnie i przerywa działanie w momencie wykrycia pierwszej niezgodności.

```
def check_compatibility(components):
    if not check_cpu_mobo(components["cpu"], components["mobo"]):
        return False
    if not check_ram_ddr(components["ram"], components["mobo"]):
        return False
    if not check_power(components["cpu"], components["gpu"], components["psu"]):
        return False
    return True
```

Takie podejście umożliwia szybkie wykrycie błędów konfiguracji oraz czytelne wskazanie przyczyny problemu użytkownikowi.

```
@router.post("/builder/save")
def save_build(components: list):
    if not check_compatibility(components):
        raise HTTPException(status_code=400)
    save_to_database(components)
```

Fragment kodu odpowiada za zapis zestawu komputerowego po wcześniejszej weryfikacji kompatybilności. Zastosowane rozwiązanie zapewnia, że do bazy danych przekazywane są wyłącznie poprawne technicznie konfiguracje.

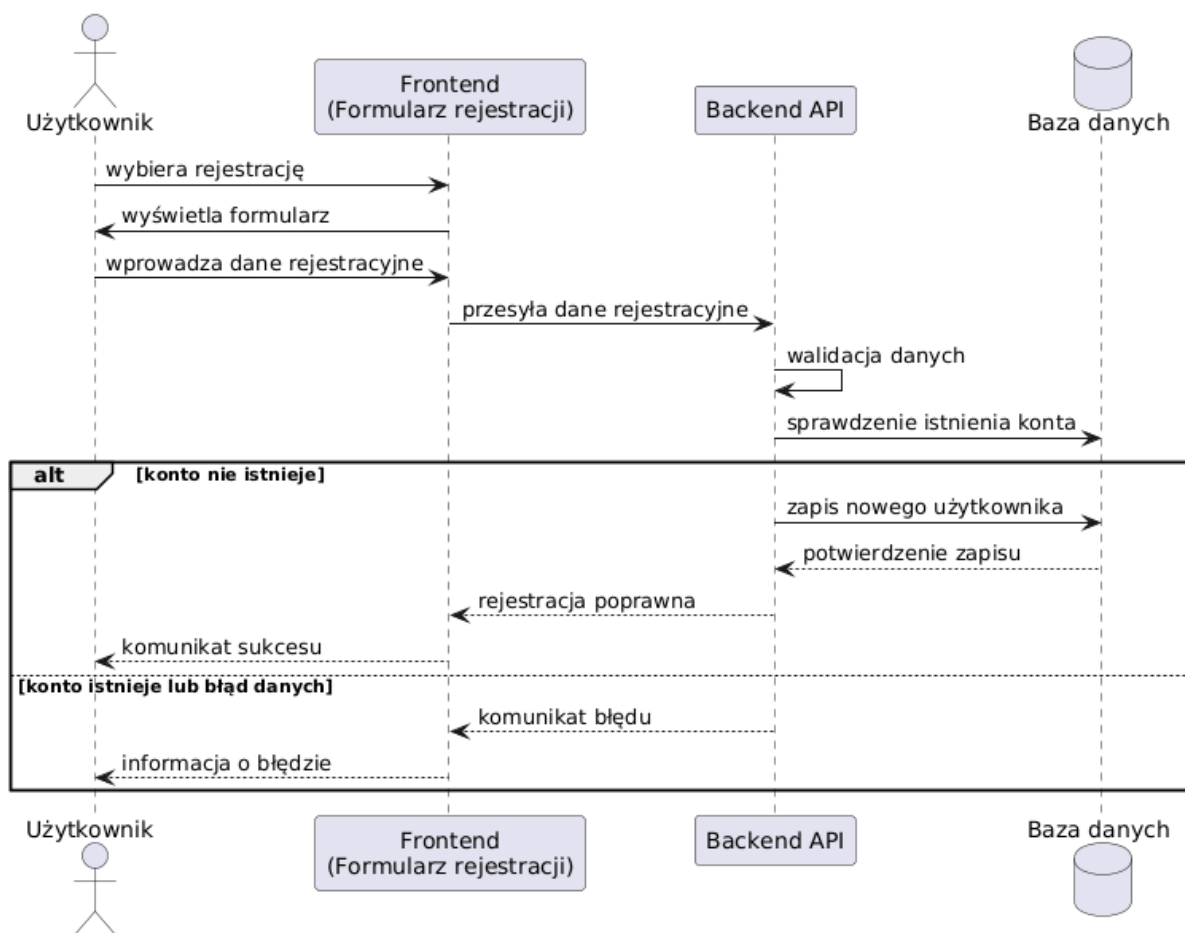
5.6 Diagramy UML i realizacja przypadków użycia

W niniejszej sekcji przedstawiono diagramy UML w postaci diagramów sekwencji, które ilustrują realizację kluczowych przypadków użycia systemu opisanych w rozdziale 3.3. Diagramy te prezentują szczegółowy przepływ danych pomiędzy użytkownikiem, warstwą frontendową, backendowym API oraz bazą danych, co pozwala na analizę dynamiki działania systemu.

5.6.1 Rejestracja użytkownika

Proces rejestracji nowego użytkownika przedstawiono na rysunku 5.3. Użytkownik inicjuje proces rejestracji poprzez interfejs frontendowy, wprowadzając wymagane dane rejestracyjne. Dane te są następnie przekazywane do backendowego API, gdzie wykonywana jest walidacja danych wejściowych oraz sprawdzenie, czy konto o podanym adresie e-mail już istnieje w bazie danych.

W przypadku braku istniejącego konta następuje zapis nowego użytkownika w bazie danych oraz zwrócenie komunikatu o poprawnej rejestracji. W przeciwnym przypadku system zwraca informację o błędzie, co zapobiega duplikacji kont użytkowników.

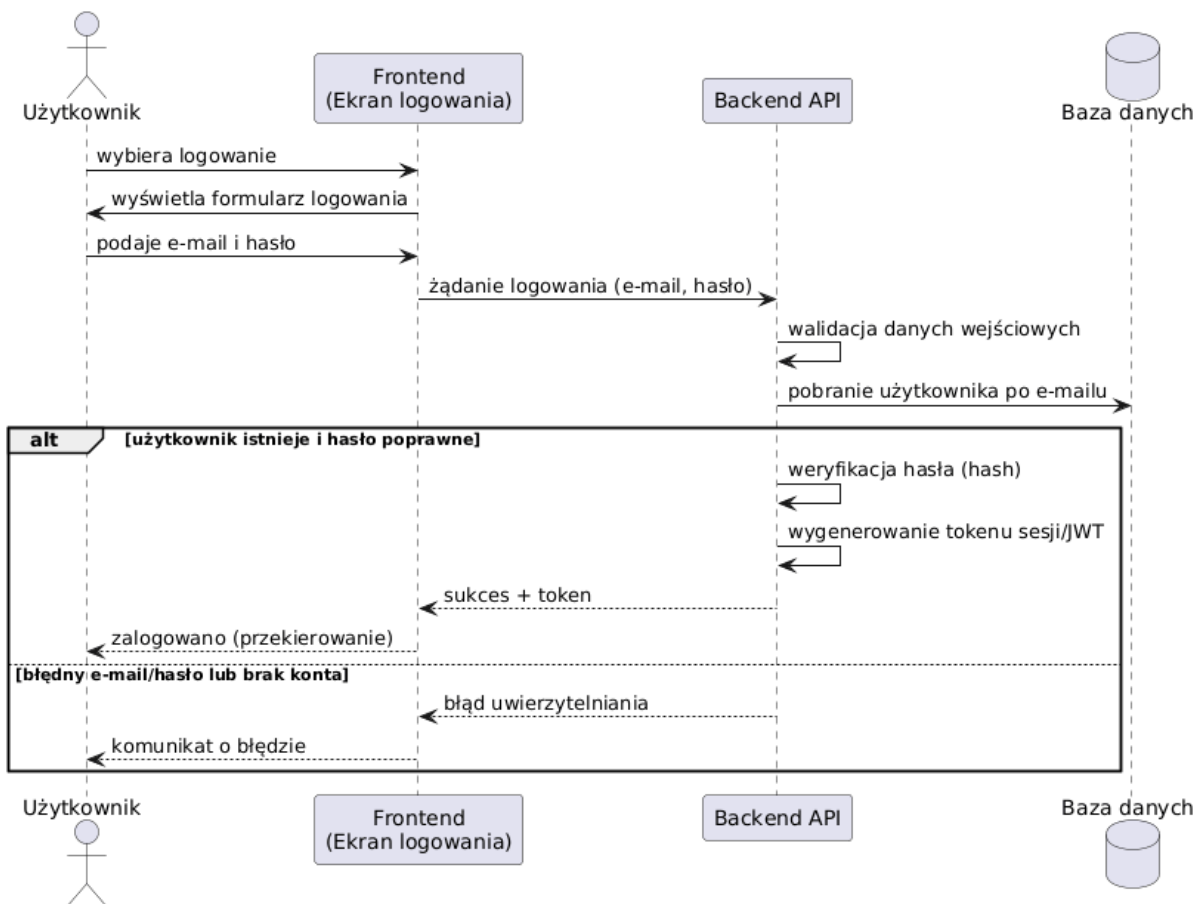


Rysunek 5.3: Diagram sekwencji procesu rejestracji użytkownika

5.6.2 Logowanie użytkownika

Diagram sekwencji logowania użytkownika przedstawiono na rysunku 5.4. Proces rozpoczyna się od wprowadzenia przez użytkownika danych uwierzytelniających w interfejsie frontendowym. Backend API dokonuje walidacji danych oraz pobiera informacje o użytkowniku z bazy danych na podstawie adresu e-mail.

W przypadku poprawnej weryfikacji hasła system generuje token sesyjny, który umożliwia dalsze korzystanie z funkcjonalności aplikacji. W sytuacji podania niepoprawnych danych uwierzytelniających użytkownik otrzymuje komunikat o błędzie.

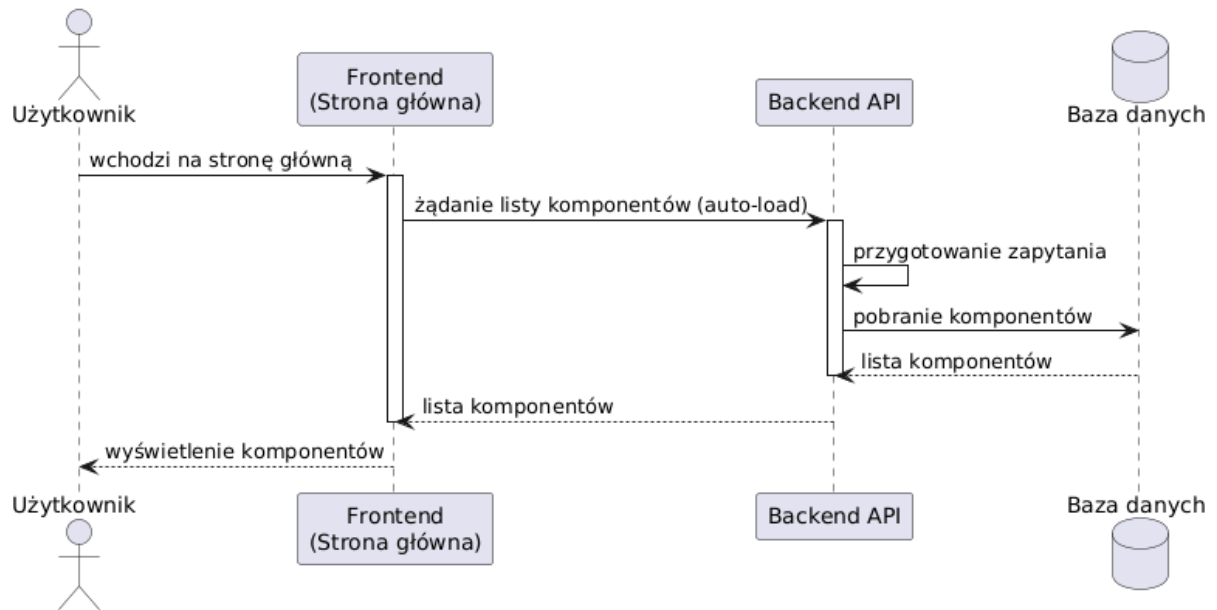


Rysunek 5.4: Diagram sekwencji procesu logowania użytkownika

5.6.3 Wyświetlanie katalogu komponentów

Proces pobierania oraz wyświetlania listy dostępnych komponentów komputerowych zaprezentowano na rysunku 5.5. Po wejściu użytkownika na stronę główną aplikacji frontend wysyła zapytanie do backendowego API w celu pobrania aktualnej listy komponentów.

Backend przygotowuje odpowiednie zapytanie do bazy danych, a następnie zwraca zestaw danych, który jest prezentowany użytkownikowi w formie katalogu produktów.

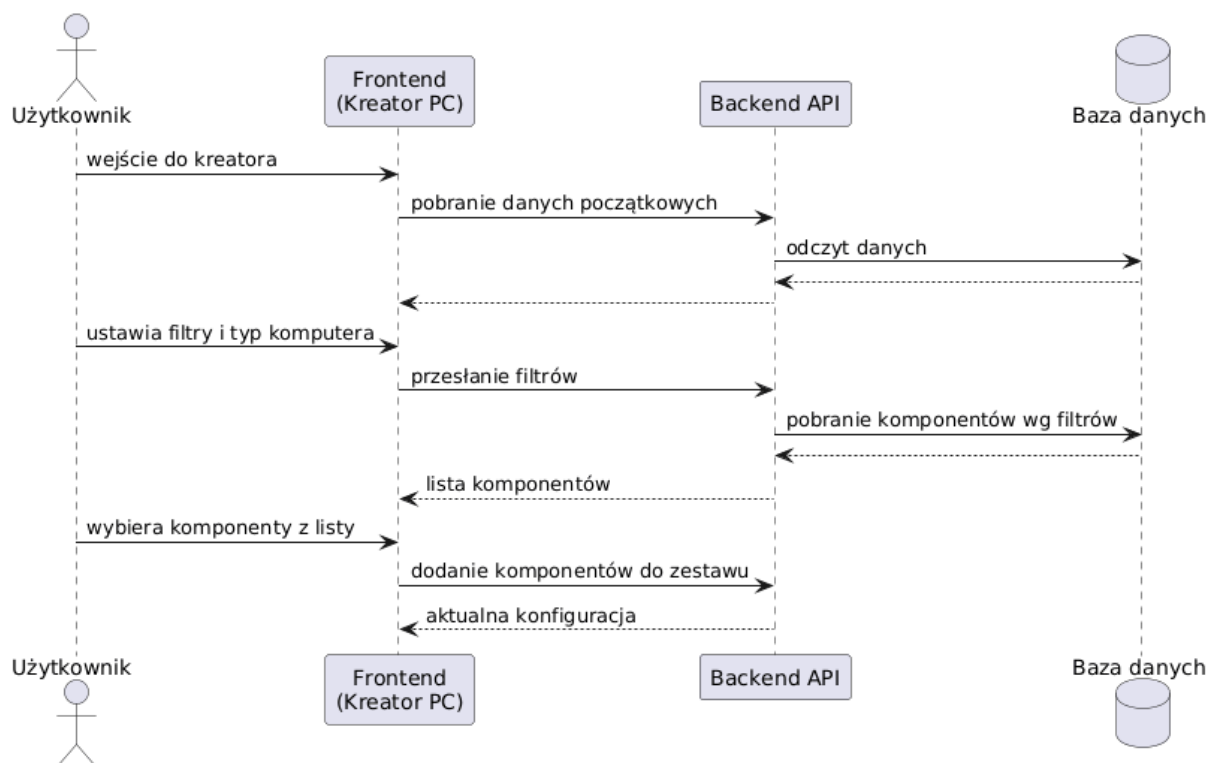


Rysunek 5.5: Diagram sekwencji wyświetlania katalogu komponentów

5.6.4 Budowa zestawu komputerowego

Proces budowy zestawu komputerowego w kreatorze PC przedstawiono na rysunku 5.6. Użytkownik wybiera komponenty z listy dostępnych produktów, a frontend przesyła informacje o aktualnej konfiguracji do backendu.

System na bieżąco aktualizuje konfigurację zestawu, umożliwiając użytkownikowi modyfikację wyboru podzespołów oraz dalszą analizę kompletności zestawu.

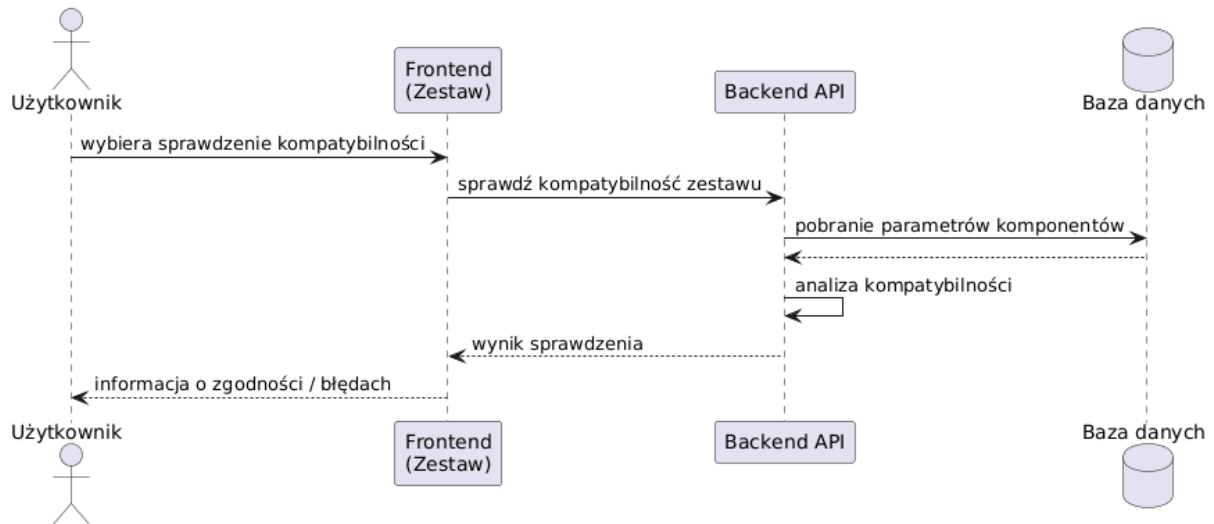


Rysunek 5.6: Diagram sekwencji procesu budowy zestawu komputerowego

5.6.5 Weryfikacja kompatybilności zestawu

Diagram sekwencji weryfikacji kompatybilności zestawu komputerowego przedstawiono na rysunku 5.7. Po zainicjowaniu sprawdzenia przez użytkownika frontend przekazuje aktualną konfigurację zestawu do backendowego API.

Backend pobiera wymagane parametry komponentów z bazy danych i wykonuje analizę kompatybilności, uwzględniając m.in. zgodność gniazd, typ pamięci RAM oraz wymagania energetyczne. Wynik analizy przekazywany jest użytkownikowi w postaci komunikatu informującego o zgodności lub wykrytych niezgodnościach.

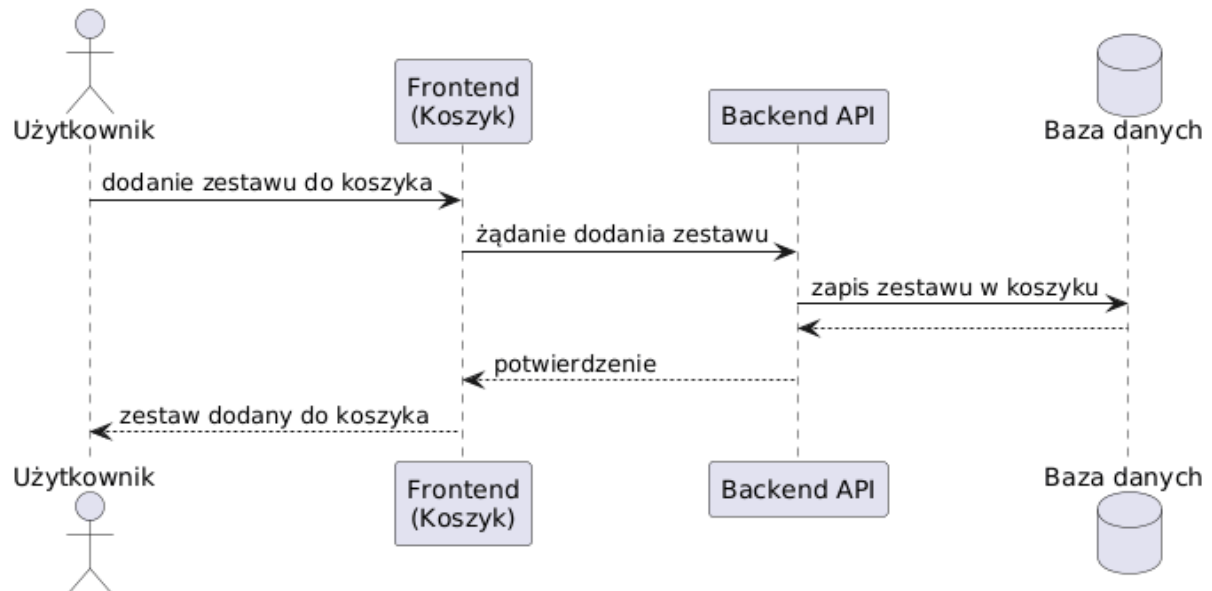


Rysunek 5.7: Diagram sekwencji weryfikacji kompatybilności zestawu

5.6.6 Dodanie zestawu do koszyka

Proces dodawania zestawu komputerowego do koszyka przedstawiono na rysunku 5.8. Po zakończeniu konfiguracji użytkownik inicjuje operację dodania zestawu do koszyka. Frontend wysyła odpowiednie żądanie do backendowego API, które zapisuje zestaw w bazie danych.

Po pomyślnym zapisie użytkownik otrzymuje potwierdzenie dodania zestawu do koszyka, co umożliwia dalszą realizację procesu zakupowego.



Rysunek 5.8: Diagram sekwencji dodawania zestawu do koszyka

Rozdział 6

Weryfikacja i walidacja

Celem niniejszego rozdziału jest weryfikacja poprawności działania zaprojektowanego systemu. Część ta szczegółowo opisuje scenariusze testowe oraz analizuje wyniki testów funkcjonalnych, potwierdzające spełnienie założonych wymagań projektowych.

6.1 Cel weryfikacji systemu

Celem weryfikacji systemu było sprawdzenie poprawności działania zaprojektowanej systemu oraz ocena stopnia spełnienia założonych wymagań funkcjonalnych. Proces testowania miał na celu weryfikację, czy wszystkie kluczowe moduły systemu działają zgodnie z przyjętymi założeniami projektowymi.

W ramach weryfikacji sprawdzono poprawność realizacji podstawowych funkcjonalności aplikacji, w tym procesu tworzenia zestawów komputerowych, obsługi koszyka oraz składania zamówień. Szczególną uwagę poświęcono mechanizmowi weryfikacji kompatybilności podzespołów, którego zadaniem jest identyfikacja konfiguracji niezgodnych technicznie.

Dodatkowo weryfikacja obejmowała ocenę stabilności działania warstwy backendowej oraz poprawności komunikacji pomiędzy frontendem a backendem. Przeprowadzone testy miały potwierdzić, że system reaguje w sposób przewidywalny na różne scenariusze użycia oraz zapewnia spójne i poprawne wyniki działania.

6.2 Metody testowania

W procesie weryfikacji systemu zastosowano testy funkcjonalne oraz testy scenariuszowe. Testy funkcjonalne polegały na sprawdzeniu poprawności działania poszczególnych funkcji aplikacji w odpowiedzi na określone dane wejściowe wprowadzane przez użytkownika.

Testy scenariuszowe zostały przeprowadzone w oparciu o realistyczne przypadki użycia systemu, obejmujące zarówno poprawne, jak i błędne konfiguracje zestawów komputerowych. Scenariusze testowe odzwierciedlały rzeczywiste działania użytkownika, takie jak dobór podzespołów, modyfikacja zestawu oraz próba utworzenia konfiguracji niezgodnej technicznie.

Testowanie realizowane było w sposób manualny wykorzystując interfejs użytkownika aplikacji. Takie podejście pozwoliło na jednoczesną weryfikację poprawności logiki biznesowej systemu oraz oceny działania warstwy frontendowej i backendowej podczas typowego użytkowania aplikacji.

6.3 Organizacja eksperymentów

Testy zostały przeprowadzone w kontrolowanym środowisku testowym, które umożliwiała weryfikację poprawności działania wszystkich kluczowych funkcjonalności systemu. Testy obejmowały zarówno warstwę frontendową, jak i backendową aplikacji oraz ich współpracę z bazą danych.

Proces testowania systemu został zorganizowany w sposób uporządkowany i powtarzalny, a poszczególne etapy eksperymentów testowych przedstawiono poniżej:

1. Przygotowanie środowiska testowego obejmujące uruchomienie backendu aplikacji oraz bazy danych zawierającej przykładowe dane produktów i użytkowników.
2. Weryfikacja poprawności procesów logowania i rejestracji użytkowników oraz pobierania danych o dostępnych produktach.
3. Przeprowadzenie testów funkcjonalnych polegających na dodawaniu komponentów do zestawu, modyfikacji konfiguracji oraz analizie reakcji systemu na zmiany dokonywane przez użytkownika.
4. Sprawdzenie działania algorytmu weryfikacji kompatybilności oraz logiki rozmytej poprzez tworzenie zarówno poprawnych, jak i niepoprawnych konfiguracji zestawów komputerowych.
5. Testowanie procesu realizacji zamówienia, obejmujące przejście przez koszyk oraz zapis danych zamówienia w bazie systemu.
6. Rejestracja wyników testów oraz porównanie uzyskanych rezultatów z oczekiwanym zachowaniem systemu.

6.4 Przypadki testowe

W niniejszym podrozdziale przedstawiono wybrane przypadki testowe wykorzystane podczas weryfikacji poprawności działania systemu. Przypadki testowe obejmują zarówno konfiguracje poprawne, jak i niepoprawne, w celu sprawdzenia reakcji systemu na różne scenariusze użytkowania.

Testy skoncentrowane były na weryfikacji mechanizmu sprawdzania kompatybilności podzespołów komputerowych, poprawności obsługi procesów użytkownika oraz realizacji podstawowych funkcjonalności aplikacji. Testowanie systemu miało charakter ręcznych testów funkcjonalnych, przeprowadzanych w środowisku deweloperskim.

Tabela 6.1: Przykładowe przypadki testowe systemu

ID	Test	Dane wejściowe	Oczekiwany wynik	Wynik
1	Zestaw kompatybilny	CPU AM5 + MOBO AM5 + RAM DDR5	Konfiguracja poprawna	Pozytywny
2	Niezgodny socket CPU–MOBO	CPU AM5 + MOBO LGA1700	Odrzucenie konfiguracji	Negatywny
3	Niezgodny standard RAM	RAM DDR5 + MOBO DDR4	Odrzucenie konfiguracji	Negatywny
4	Zbyt długa karta graficzna	GPU > maks. długość obudowy	Odrzucenie konfiguracji	Negatywny
5	Zbyt słaby zasilacz	PSU < zapotrzebowanie CPU/GPU	Odrzucenie konfiguracji	Negatywny
6	Zgodne chłodzenie CPU	CPU + COOLER zgodny z socketem	Konfiguracja poprawna	Pozytywny
7	Poprawna rejestracja	Poprawne dane użytkownika	Rejestracja zakończona sukcesem	Pozytywny
8	Błędne logowanie	Niepoprawne hasło	Odmowa dostępu	Negatywny

6.5 Wyniki testów

Tabela 6.2: Przykładowe wyniki testów kompatybilności zestawów komputerowych.

ID	Test	Dane wejściowe	Problem	Wynik
1	CPU + płyta główna	Ryzen 5 7600 + B650	Brak	Kompatybilne
2	CPU + płyta główna	i5-12600K + B550	Różne sockety	Niekompatybilne
3	RAM + płyta główna	DDR5 6000 + B460	RAM DDR5 + MOBO DDR4	Niekompatybilne
4	GPU + obudowa	RTX 4070 (330 mm) + obudowa 300 mm	Zbyt długie GPU	Niekompatybilne
5	Zasilacz + CPU/GPU	PSU 400W + RTX 3080	Zbyt mała moc	Niekompatybilne
6	Chłodzenie + socket	Cooler AM4 + CPU LGA1700	Niezgodne typy chłodzenia	Niekompatybilne
7	Zestaw kompletny	CPU + GPU + RAM + Mobo + PSU zgodne	Brak	Kompatybilne
8	Zestaw błędny	3 niezgodności (DDR, TDP, GPU)	Złożona	Niekompatybilne

6.6 Wykryte i usunięte błędy

W trakcie realizacji projektu oraz procesu testowania systemu wykryto kilka nieprawidłowości związanych głównie z obsługą danych oraz komunikacją pomiędzy poszczególnymi warstwami aplikacji. Błędy te miały charakter implementacyjny i zostały usunięte na etapie prac rozwojowych.

Jednym z napotkanych problemów były nieprawidłowości w przeliczaniu wartości cenowych produktów, wynikające z obsługi typu numeric po stronie bazy danych oraz jego mapowania w warstwie backendowej. W niektórych przypadkach wartości cenowe były błędnie interpretowane lub zaokrąglane. Problem został rozwiązany poprzez doprecyzowanie typów danych w modelach oraz ujednolicenie sposobu przetwarzania wartości liczbowych w aplikacji.

Dodatkowo wykryto błędy związane z niejednoznacznym mapowaniem typów danych w bibliotece ORM, co prowadziło do niepoprawnej walidacji wybranych parametrów technicznych podzespołów. Po analizie problemu wprowadzono korekty w definicjach modeli danych oraz regułach walidacyjnych.

W początkowej fazie testów pojawiły się również problemy konfiguracyjne związane z komunikacją pomiędzy frontendem a backendem, w tym błędy wynikające z nieprawidłowych ustawień środowiskowych. Po poprawieniu konfiguracji aplikacji oraz ujednoliceniu parametrów połączeń problemy te zostały całkowicie wyeliminowane.

Usunięcie wskazanych błędów pozwoliło na uzyskanie stabilnego działania systemu oraz poprawne funkcjonowanie mechanizmów walidacji i kompatybilności.

6.7 Ocena poprawności działania systemu

Na podstawie przeprowadzonych testów oraz uzyskanych wyników można stwierdzić, że zaprojektowany system działa poprawnie i spełnia założone wymagania funkcjonalne. Kluczowe mechanizmy aplikacji, w szczególności proces tworzenia zestawów komputerowych oraz weryfikacji kompatybilności podzespołów, funkcjonują zgodnie z przyjętymi założeniami projektowymi.

System prawidłowo identyfikuje konfiguracje niezgodne technicznie oraz poprawnie obsługuje przypadki poprawnych zestawów, umożliwiając ich dalszą realizację w procesie zakupowym. Przeprowadzone testy potwierdziły również stabilność działania warstwy backendowej oraz poprawność komunikacji pomiędzy frontendem a backendem.

Uzyskane wyniki testów potwierdzają, że działanie systemu jest zgodne z wymaganiami funkcjonalnymi oraz нефunkcjonalnymi określonymi w rozdziale trzecim pracy. Przeprowadzone przypadki testowe wykazały poprawną realizację kluczowych funkcji systemu, takich jak rejestracja i logowanie użytkowników, przeglądanie katalogu produktów, tworzenie zestawów komputerowych oraz automatyczna weryfikacja kompatybilności podzespołów.

System spełnił również założenia нефunkcjonalne w zakresie stabilności, czytelności komunikatów oraz przewidywalności działania. W szczególności mechanizm weryfikacji kompatybilności reagował w sposób kontrolowany i jednoznaczny na konfiguracje niezgodne technicznie, co potwierdza poprawność przyjętych założeń projektowych.

Rozdział 7

Podsumowanie i wnioski

Celem niniejszego rozdziału jest podsumowanie rezultatów uzyskanych podczas realizacji pracy inżynierskiej oraz ocena stopnia spełnienia założonych celów. Przedstawiono również trudności napotkane w trakcie realizacji projektu oraz możliwe kierunki dalszego rozwoju systemu.

7.1 Realizacja celów pracy

Głównym celem pracy było zaprojektowanie oraz implementacja systemu sklepu komputerowego z kreatorem zestawów PC, umożliwiającego użytkownikowi dobór kompatybilnych podzespołów komputerowych. Cel ten został osiągnięty poprzez opracowanie kompletnej architektury aplikacji obejmującej warstwę frontendową, backendową oraz relacyjną bazę danych.

W ramach realizacji projektu zaprojektowano znormalizowany model danych, zaimplementowano backend odpowiedzialny za logikę biznesową oraz mechanizmy weryfikacji kompatybilności podzespołów, a także przygotowano interfejs użytkownika umożliwiający interaktywne tworzenie zestawów komputerowych. Zrealizowane funkcjonalności odpowiadają na problem przedstawiony w rozdziale pierwszym, polegający na braku narzędzi umożliwiających automatyczną analizę zgodności komponentów w klasycznych sklepach internetowych.

7.2 Ocena otrzymanych rezultatów

Otrzymane rezultaty należy ocenić pozytywnie. Zaimplementowany system działa zgodnie z przyjętymi założeniami projektowymi i umożliwia poprawne tworzenie zestawów komputerowych wraz z ich weryfikacją pod kątem kompatybilności. Najważniejsze funkcje systemu, takie jak dobór podzespołów, analiza zgodności technicznej oraz obsługa koszyka i zamówień, zostały przetestowane i potwierdziły swoją poprawność.

Do mocnych stron rozwiązania należy zaliczyć przejrzystą architekturę, czytelny interfejs użytkownika oraz elastyczny model danych umożliwiający dalszą rozbudowę systemu. Szczególną wartość stanowi mechanizm sprawdzania kompatybilności, który pozwala na eliminację błędnych konfiguracji już na etapie tworzenia zestawu. System może być z powodzeniem wykorzystywany jako narzędzie wspomagające proces wyboru komponentów komputerowych.

7.3 Trudności napotkane podczas realizacji

Podczas realizacji projektu napotkano szereg trudności natury technicznej. Jednym z głównych wyzwań było zaprojektowanie uniwersalnego modelu danych, który umożliwiłby przechowywanie parametrów technicznych różnych typów podzespołów komputerowych, charakteryzujących się odmiennymi właściwościami.

Trudności pojawiły się również na etapie implementacji logiki weryfikacji kompatybilności, wymagającej uwzględnienia wielu zależności pomiędzy komponentami, takich jak zgodność gniazd procesora, standardów pamięci RAM czy ograniczeń wynikających z obudowy. Dodatkowym wyzwaniem była integracja warstwy frontendowej z backendem oraz zapewnienie poprawnej komunikacji pomiędzy poszczególnymi modułami systemu.

7.4 Możliwe kierunki dalszego rozwoju

Zaprojektowany system może stanowić podstawę do dalszego rozwoju. Jednym z możliwych kierunków jest rozbudowa mechanizmów logiki rozmytej poprzez zwiększenie liczby kategorii zestawów oraz bardziej precyzyjne reguły klasyfikacji. Możliwe jest również wprowadzenie automatycznego doboru podzespołów na podstawie zadanego budżetu użytkownika.

Dalszy rozwój systemu mógłby obejmować integrację z zewnętrznymi interfejsami API sklepów internetowych, wprowadzenie panelu administracyjnego do zarządzania katalogiem produktów, a także przygotowanie wersji mobilnej aplikacji. Potencjalnym kierunkiem rozwoju jest również zastosowanie algorytmów uczenia maszynowego w celu generowania bardziej zaawansowanych rekomendacji.

7.5 Wnioski końcowe

Zrealizowany projekt potwierdził możliwość stworzenia systemu, który w sposób zautomatyzowany wspiera użytkownika w procesie doboru kompatybilnych podzespołów komputerowych. Opracowane rozwiązanie łączy funkcjonalności typowe dla systemów e-commerce z mechanizmami analizy technicznej zestawów, co stanowi jego istotną wartość użytkową.

Podsumowując, praca spełniła założone cele projektowe i potwierdziła zdobyte w trakcie studiów umiejętności z zakresu projektowania baz danych, tworzenia aplikacji webowych oraz analizy problemów inżynierskich. Zaproponowany system jest rozwiązaniem kompletnym, spójnym i praktycznie użytecznym, mogącym stanowić podstawę do dalszego rozwoju i wdrożeń.

Bibliografia

- [1] *ESLint - Find and fix problems in your JavaScript code.* 2024. URL: <https://eslint.org/> (dostęp 1.02.2025).
- [2] *FastAPI - High performance, easy to learn, fast to code.* 2024. URL: <https://fastapi.tiangolo.com/> (dostęp 5.02.2025).
- [3] *Git - Distributed Version Control System.* 2024. URL: <https://git-scm.com/> (dostęp 12.01.2025).
- [4] *GitHub - Where the world builds software.* 2024. URL: <https://github.com/> (dostęp 14.01.2025).
- [5] *Microsoft Azure - Cloud Computing Services.* 2024. URL: <https://azure.microsoft.com/> (dostęp 8.02.2025).
- [6] *Migrate PostgreSQL to Azure Database for PostgreSQL using Azure Database Migration Service.* 2025. URL: <https://learn.microsoft.com/en-us/azure/dms/tutorial-postgresql-azure-postgresql-online-portal> (dostęp 12.10.2025).
- [7] *Morele.net - sklep komputerowy.* 2025. URL: <https://www.morele.net/> (dostęp 17.11.2025).
- [8] *Node.js - JavaScript runtime.* 2024. URL: <https://nodejs.org/> (dostęp 20.01.2025).
- [9] *npm - Node Package Manager.* 2024. URL: <https://www.npmjs.com/> (dostęp 22.01.2025).
- [10] *Oracle SQL Developer Data Modeler.* 2024. URL: <https://www.oracle.com/database/sqldeveloper/technologies/sql-data-modeler/> (dostęp 30.01.2025).
- [11] *PC Part Dataset - Public Hardware Dataset for Computer Components.* 2025. URL: <https://github.com/docyx/pc-part-dataset> (dostęp 11.09.2025).
- [12] *PostgreSQL - The World's Most Advanced Open Source Relational Database.* 2024. URL: <https://www.postgresql.org/> (dostęp 18.01.2025).
- [13] *psycopg2 - PostgreSQL adapter for Python.* 2024. URL: <https://www.psycopg.org/> (dostęp 25.01.2025).
- [14] *Pydantic - Data validation using Python type hints.* 2024. URL: <https://docs.pydantic.dev/> (dostęp 4.02.2025).

- [15] *Python Programming Language*. 2024. URL: <https://www.python.org/> (dostęp 15.01.2025).
- [16] *React - A JavaScript library for building user interfaces*. 2024. URL: <https://react.dev/> (dostęp 12.02.2025).
- [17] *SQLAlchemy - The Python SQL Toolkit and ORM*. 2024. URL: <https://www.sqlalchemy.org/> (dostęp 6.02.2025).
- [18] *Tailwind CSS - A Utility-First CSS Framework*. 2024. URL: <https://tailwindcss.com/> (dostęp 7.02.2025).
- [19] *TypeScript - Typed JavaScript at Any Scale*. 2024. URL: <https://www.typescriptlang.org/> (dostęp 28.01.2025).
- [20] *Uvicorn - A lightning-fast ASGI server*. 2024. URL: <https://www.uvicorn.org/> (dostęp 2.02.2025).
- [21] *Vite - Next Generation Frontend Tooling*. 2024. URL: <https://vitejs.dev/> (dostęp 3.02.2025).
- [22] *X-kom - sklep komputerowy*. 2025. URL: <https://www.x-kom.pl/> (dostęp 17.11.2025).

Dodatki

Spis skrótów i symboli

API interfejs programistyczny aplikacji (ang. *Application Programming Interface*)

CPU procesor centralny (ang. *Central Processing Unit*)

CRUD podstawowe operacje na danych: tworzenie, odczyt, modyfikacja, usuwanie (ang. *Create, Read, Update, Delete*)

ERD diagram związków encji (ang. *Entity–Relationship Diagram*)

GPU procesor graficzny (ang. *Graphics Processing Unit*)

HTTP protokół komunikacji klient–serwer (ang. *HyperText Transfer Protocol*)

JSON format danych tekstowych (ang. *JavaScript Object Notation*)

ORM mapowanie obiektowo–relacyjne (ang. *Object–Relational Mapping*)

RAM pamięć operacyjna (ang. *Random Access Memory*)

REST styl architektury usług sieciowych (ang. *Representational State Transfer*)

SQL język zapytań do baz danych (ang. *Structured Query Language*)

SSD dysk półprzewodnikowy (ang. *Solid State Drive*)

TDP maksymalna ilość ciepła odprowadzana przez układ (ang. *Thermal Design Power*)

UI interfejs użytkownika (ang. *User Interface*)

UML zunifikowany język modelowania (ang. *Unified Modeling Language*)

Spis rysunków

2.1	Przykładowe funkcje przynależności dla kategorii cenowych zestawu komputerowego	8
4.1	Widok strony głównej aplikacji	23
4.2	Filtrowanie produktów z katalogu	24
4.3	Wybór produktów z przefiltrowanego katalogu	24
4.4	Widok koszyka użytkownika z dodanymi produktami	25
4.5	Poprawne logowanie użytkownika	25
4.6	Komunikat o niepoprawnych danych logowania	26
4.7	Widok kreatora zestawów komputerowych	27
4.8	Komunikat o niezgodności podzespołów	28
5.1	Schemat ideowy działania aplikacji	32
5.2	Diagram ERD bazy danych	40
5.3	Diagram sekwencji procesu rejestracji użytkownika	61
5.4	Diagram sekwencji procesu logowania użytkownika	62
5.5	Diagram sekwencji wyświetlania katalogu komponentów	63
5.6	Diagram sekwencji procesu budowy zestawu komputerowego	64
5.7	Diagram sekwencji weryfikacji kompatybilności zestawu	65
5.8	Diagram sekwencji dodawania zestawu do koszyka	65

Spis tabel

5.1	Struktura tabeli Produkty	43
5.2	Struktura tabeli Klienci	44
5.3	Struktura tabeli Koszyk	45
5.4	Struktura tabeli Zamówienia	46
5.5	Struktura tabeli Zestawy	47
5.6	Struktura tabeli GPU	48
5.7	Struktura tabeli CPU	50
5.8	Struktura tabeli MOBO	51
5.9	Struktura tabeli RAM	53
5.10	Struktura tabeli PSU	54
5.11	Struktura tabeli COOLER	55
5.12	Struktura tabeli DYSK	56
6.1	Przykładowe przypadki testowe systemu	69
6.2	Przykładowe wyniki testów kompatybilności zestawów komputerowych. . .	70